

Probabilistic Programming with Programmable Divide-Conquer-Combine Inference on Modern Hardware

MARKUS BÖCK, TU Wien, Austria

JÜRGEN CITO, TU Wien, Austria

Universal probabilistic programming languages (PPLs) enable the specification of models with stochastic support structure. Posterior inference is notoriously hard for this class of models and remains difficult to accelerate on modern hardware. In response to these challenges, we introduce UPIX – the first probabilistic programming system that realises the divide-conquer-combine (DCC) inference algorithm as a framework. In UPIX, a model expressed in a universal PPL is automatically split into multiple sub-models with static support structure, which are then compiled with JAX for execution on accelerator hardware. The system allows extensive customisation of inference algorithms by incorporating established concepts from programmable inference literature. To evaluate our system, we implemented two existing DCC algorithms in UPIX and instantiated three novel algorithms. We show that our implementation can result in better approximation quality compared to existing approaches by achieving up to 960 times more computation within the same time budget. On machines with up to 64 CPU cores and 8 GPU devices, we demonstrate that UPIX enables the scaling of inference algorithms to workloads that are impractically slow for CPUs and prior methods.

CCS Concepts: • **Mathematics of computing** → **Bayesian computation**; **Statistical software**; • **Software and its engineering** → *Just-in-time compilers*.

Additional Key Words and Phrases: probabilistic programming, stochastic support, divide-conquer-combine, programmable inference, GPU-acceleration

ACM Reference Format:

Markus Böck and Jürgen Cito. 2026. Probabilistic Programming with Programmable Divide-Conquer-Combine Inference on Modern Hardware. *Proc. ACM Program. Lang.* (in submission), -, Article 0 (January 2026), 36 pages. <https://doi.org/10.0000/0000000>

1 Introduction

Probabilistic programming languages (PPLs) enable the specification of probabilistic models as programs and automate Bayesian inference. In their implementations, they have to trade off expressivity and inference efficiency.

On one side, there are PPLs like PyMC [52] or Stan [9] which restrict the class of programs to models with fixed and finite-dimensional support structure. This allows efficient model representation and optimised implementations of inference algorithms like HMC [6], ADVI [26], or NUTS [22]. Even more, some probabilistic systems in this category, like TensorflowProbability [27], BlackJAX [8], or NumPyro [45], build on the JAX numerical computing library to run inference on CPUs, GPUs, and TPUs via just-in-time (JIT) compilation.

On the other side, there are so-called *universal* PPLs like Gen [13], Turing [15], or Anglican [54], which embed their probabilistic constructs in a Turing-complete programming language. To support the resulting larger class of models, they have to rely on general-purpose inference algorithms

Authors' Contact Information: Markus Böck, TU Wien, Vienna, Austria, markus.h.boeck@tuwien.ac.at; Jürgen Cito, TU Wien, Vienna, Austria, juergen.cito@tuwien.ac.at.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2475-1421/2026/1-ART0

<https://doi.org/10.0000/0000000>

which often can be inefficient. To overcome this problem, these PPLs typically allow the user to customise the inference algorithms for the specific model at hand. This type of customisation, which came to be known as *programmable inference*, is actively researched and developed [5, 12, 13, 30, 35].

Universal PPLs gained popularity and research interest, because models with stochastic support structure can be easily expressed in them. Such models include mixture models with an unknown number of components [44, 48], kernel and program induction models [50, 51], "open-universe" models [38, 58], statistical phylogenetics models [49], models based on physical simulations [3], and many Bayesian non-parametric models [20, 33, 56].

However, the dynamic nature of programmable inference algorithms for models defined in a universal PPL makes GPU or TPU parallelisation difficult, given that such hardware is optimised for single-instruction multiple-data (SIMD) workloads. To the best of our knowledge, there exists no system that 1) makes it easy to parallelise inference on GPUs or TPUs, 2) enables the specification of models in a universal PPL, and 3) implements a programmable inference machinery. In this work, we propose to bridge this gap by building a system following the *divide-conquer-combine* (DCC) approach introduced by Zhou et al. [59] in 2020.

Conceptionally, the DCC approach is rather simple: In the *divide step*, a model specified in a universal PPL is split up into multiple sub-models with fixed and finite-dimensional support structure. In the *conquer step*, inference is performed on the sub-models by leveraging optimised algorithms. Lastly, in the *combine step*, all results are combined in a consistent way to approximate the posterior distribution of the full model. However, in practice, each step comes with challenges:

- **Divide.** It is difficult to automatically split up a probabilistic program into sub-models. Existing approaches either rely on user-annotation or consider only the simpler case in which the program is split up by conditioning on the values of discrete variables.
- **Conquer.** The derived sub-models may still pose challenging inference problem rendering black-box approaches infeasible.
- **Combine.** To combine the results, we have to weigh the inference result of each sub-model proportional to its contribution to the full posterior. This amounts to computing the *normalisation constant* accurately for each sub-model – a notoriously difficult problem.

This work. We present UPIX – a probabilistic programming system, which solves the above challenges by implementing the DCC approach for a universal PPL as a framework with programmable inference in JAX: In short, a JAX program transformation is developed to split up the probabilistic programs into sub-models automatically. Next, the system provides constructs to customise *both* the inference routine and the normalisation constant computation for each sub-model making the conquer and divide step fully programmable. Further, as a product of implementing the system in JAX, inference in UPIX can greatly leverage vectorisation on CPUs, GPUs, and TPUs, as well as parallelisation across multiple devices. Lastly, through its abstractions UPIX enables the development of new DCC-based inference algorithms.

Contributions. This paper makes the following contributions:

- UPIX [1] – the first probabilistic programming system that realises the DCC approach as a framework and implements a programmable inference machinery to run inference on CPUs, GPUs, or TPUs for models specified in a universal PPL.
- A sound program transformation to automatically identify and compile sub-models.
- Five instantiations of the framework yielding *three novel DCC-based algorithms*: a Markov chain Monte Carlo approach as in the original DCC publication [59], Support Decomposition Variational Inference (SDVI) [46], a Reversible Jump MCMC inspired approach, a Sequential Monte Carlo based approach, and a Variable Elimination based approach for discrete models.

- An empirical evaluation which compares UPIX to existing work on challenging models showing that UPIX achieves up to 960 times more computation within the same time budget on consumer-grade CPUs, which can substantially improve approximation quality.
- An investigation of the scaling properties of UPIX on modern hardware (up to 64 CPU cores and 8 GPU devices), demonstrating that inference algorithms can be scaled to workloads that are impractically slow for CPUs and existing approaches.

2 Overview

In this section, we introduce and motivate concepts relevant to this work, parts of which will be formalised in Section 3. To begin with, *probabilistic programming languages* (PPLs) are programming languages equipped with constructs for declaring random variables and conditioning on observed data. In this work, we focus on *universal PPLs* which are PPLs embedded in a Turing-complete language that supports stochastic control flow and recursion [17]. In particular, we consider a language that allows the user to declare a random variable by linking a dynamically computed label – the *address* – to a dynamically computed distribution in a *sample statement*. Conditioning of the model is achieved by fixing the values of selected addresses to observed data.

In Figure 1 on the left, we show how the mixture model

$$B \sim \text{Bernoulli}(0.5), \quad z \sim \begin{cases} \text{Normal}(-3, 1) & \text{if } B = 1, \\ \text{Uniform}(1, 4) & \text{otherwise.} \end{cases}, \quad y \sim \text{Normal}(z, 2)$$

may be implemented with stochastic branching in a universal PPL.

```
def mixture_1():
    B = sample("B", Bernoulli(0.5),
              branching=True)
    if B == 1:
        z = sample("z", Normal(-3, 1))
    else:
        z = sample("z", Uniform(1, 4))
    sample("y", Normal(z, 2), observed=2)

def mixture_2():
    U = sample("U", Uniform(0, 1))
    if U > 0.5:
        z = sample("z1", Normal(-3, 1))
    else:
        z = sample("z2", Uniform(1, 4))
    sample("y", Normal(z, 2), observed=2)
```

Fig. 1. Two mixture models implemented in a universal PPL with stochastic branching. We highlight the annotations required by *prior* DCC implementations to identify SLPs in yellow.

In this work, the domain of probabilistic programs is defined with *traces* – mappings from addresses of latent random variables to their values. We interpret a probabilistic program as a function p mapping traces to their (unnormalised) posterior density. The function p is only well-defined for traces that are compatible with executions of the program. The *support* of a probabilistic program is the set of traces which map to a positive density. For instance, for the mixture model, we have $p(\{B \mapsto 0, z \mapsto 2.3\}) = \text{pdf}_{\text{Bernoulli}(0.5)}(0) \cdot \text{pdf}_{\text{Uniform}(1,4)}(2.3) \cdot \text{pdf}_{\text{Normal}(2.3,2)}(2) \approx 0.0329$.

The great expressivity of universal PPLs comes at the cost of making automated inference more challenging. The Divide-Conquer-Combine (DCC) approach [59] proposes to solve this problem by splitting up the model support, given by a set of traces $\mathcal{T}$, into an (typically) infinite number of disjoint subsets $\bigsqcup_k \mathcal{T}_k = \mathcal{T}$ such that the support structure on $\mathcal{T}_k$ is static and finite-dimensional (essentially isomorphic to some $\mathbb{Z}^m \times \mathbb{R}^n$ measure space). This *divide step* splits the model into multiple sub-models such that $p(\text{tr}) = \sum_k p_k(\text{tr})$ and $p_k(\text{tr}) = [\text{tr} \in \mathcal{T}_k] \cdot p(\text{tr})$. In the context of probabilistic programming, these sub-models are referred to as *straight-line programs* (SLPs) as they are free from universal language features like stochastic control flow and recursion.

For the example mixture model, the two disjoint sub-models $p_1(\text{tr}) = [\text{tr}(\text{B}) = 1] \cdot p(\text{tr})$ and $p_2(\text{tr}) = [\text{tr}(\text{B}) \neq 1] \cdot p(\text{tr})$ are given by the SLPs below, which return the log-density along with a boolean value indicating whether the trace belongs to the sub-model.

```
def mixture_1_SLP1(tr):
    lp = 0.0
    lp += Bernoulli(0.5).log_prob(tr["B"])
    lp += Normal(-3,1).log_prob(tr["z"])
    lp += Normal(z,2).log_prob(2)
    return lp, tr["B"] == 1

def mixture_1_SLP2(tr):
    lp = 0.0
    lp += Bernoulli(0.5).log_prob(tr["B"])
    lp += Uniform(1,4).log_prob(tr["z"])
    lp += Normal(z,2).log_prob(2)
    return lp, tr["B"] != 1
```

It is desirable for systems implementing the DCC approach to be capable of automatically finding and compiling the associated SLP functions. However, the only existing implementations [46, 59] rely on user-annotations to mark discrete branching variables or require the model to be rewritten such that the set of sample addresses encountered in a program run uniquely identifies the SLP when branching depends on continuous variables, see Figure 1. By building UPIX on JAX [14], a *tracing just-in-time* (JIT) compiler for Python, we are able to fully automate SLP generation *without user-annotation*, see Section 3.4.

Furthermore, we consider a previously unrecognised source for stochastic model support in DCC methods. To illustrate, we show a Gaussian Mixture Model (GMM) with an unknown number of components κ in Figure 2. In this program, the shapes of the arrays passed as arguments to the distribution objects of `w`, `mus`, and `vars` are dynamically determined by the value of κ , thereby inducing a stochastic support whose structure varies with κ . Our JAX implementation is able to identify the SLPs fully automatically even in this case. In contrast, prior implementations require the annotation of κ with `branching=True` or modifying the addresses, e.g., `"w"+str(K)`.

```
def gmm(ys: jax.Array):
    N = ys.shape[0]
    K = sample("K", Poisson(lam=1)) + 1 # influences the shape of w, mus, and vars
    w = sample("w", Dirichlet(jnp.full((K,), delta)))
    mus = sample("mus", Normal(jnp.full((K,), xi), jnp.full((K,), 1/jnp.sqrt(kappa))))
    vars = sample("vars", InverseGamma(jnp.full((K,), alpha), jnp.full((K,), beta)))
    zs = sample("zs", Categorical(jax.lax.broadcast(w, (N,))))
    sample("ys", Normal(mus[zs], jnp.sqrt(vars[zs])), observed=ys)
```

Fig. 2. Gaussian Mixture Model implemented in UPIX making use of dynamic array shapes.

The GMM also demonstrates the benefits and challenges of the conquer and combine steps of DCC. In the *conquer step*, efficient inference algorithms for the static and finite-dimensional sub-models are used to approximate the distribution specified by the unnormalised densities $p_k(\text{tr}) = [\text{tr}(\text{K}) = k] \cdot p(\text{tr})$. Let $\hat{p}_k$ denote the density of such approximations. While easier than inference for the full model, inference for the sub-models is already challenging and black-box approaches as realised in existing DCC implementations perform poorly for complex models. Instead, we propose to use established programmable inference constructs [5, 12, 13, 30] for these sub-problems. For instance, for the GMM, an MCMC approach based on well-known Gibbs kernels [48] can be readily programmed as proposal distributions for a Metropolis-Hastings algorithm. Notably, these Gibbs kernels only exist for the finite-dimensional sub-models with fixed κ , not for the full Gaussian mixture model.

A drawback of splitting the model into multiple sub-problems is the need to allocate computational resources effectively among high-probability SLPs. While Zhou et al. [59] and Reichelt et al. [46] have proposed general resource allocation techniques for DCC, we leave room in UPIX to customise resource allocation programmatically.

Finally, in the *combine step* of DCC, the estimates of the sub-models have to be combined in a consistent manner with $\hat{p} \approx \sum_k \hat{w}_k \hat{p}_k$. In the original formulation of DCC, the weight $\hat{w}_k = \hat{Z}_k / \sum_j \hat{Z}_j$ for $\hat{p}_k$ is computed by estimating the *normalisation constants* or *marginal likelihoods* $\hat{Z}_k = \int_{\mathcal{T}_k} p(\text{tr}) d\text{tr}$ with a variant of importance sampling. Unfortunately, this integral becomes very challenging for complex models, and we found that general-purpose estimation techniques like importance sampling are insufficient. Again, we propose to customise this estimation programmatically in UPIX for which we provide programmable inference constructs. For instance, as we will elaborate in Section 4.3, we compute the SLP weight $\hat{w}_k$ for the GMM by implementing a *reversible-jump kernel* that allows us to estimate the probability of "switching" between sub-models in the full posterior.

Lastly, it is apparent that a generic DCC framework with programmable inference has many moving parts and assessing the correctness of an inference routine for a given model is challenging. Nevertheless, we can state following necessary assumptions (adapted from Zhou et al. [59]):

- (A1) The number of sub-models is finite.
- (A2) For each sub-model, the density estimate $\hat{p}_k$ of the local inference routine converges to the true sub-model density p_k .
- (A3) For each sub-model, the local inference routine provides an estimate that can be used to build a consistent estimate for the weight $w_k = Z_k / Z$, where $Z = \int_{\mathcal{T}} p(\text{tr}) d\text{tr}$ is the marginal likelihood of the full model.
- (A4) The number of iterations to discover all sub-models is almost surely finite.
- (A5) In the limit, each sub-model with $Z_k > 0$ is visited an infinite number of times.

Assumption (A1) seems restrictive, but is only a technicality. Even though we can support an infinite number of SLPs, for theoretical correctness, we would need to lump together all but the finitely many most relevant SLPs to a single sub-model (with stochastic support structure) and run inference on it. In practice, we assume that all SLPs that were not selected for local inference only contribute in a negligible way to the full posterior and do not run inference on them at all. Assumptions (A2) and (A3) are typically guaranteed by the soundness of the used local inference routines. Lastly, assumptions (A4) and (A5) depend on the employed exploration strategy. We will describe proven strategies in the following sections.

3 The UPIX System

3.1 Language

UPIX allows the specification of probabilistic models in a universal PPL embedded in Python. Our syntax is adapted from Pyro, with sample statements invoking the abstract function

```
def sample(address:str, distribution:Distribution, [observed:Array]) -> Array (1)
```

where `address` is a unique identifier for a random variable that is distributed according to `argument distribution`, and the `observed` argument is optional for incorporating observed data. A model is given by a Python function annotated with the decorator `@model` and includes sample statements mixed with arbitrary Python code that modifies JAX arrays. Importantly, Python control flow, like if statements and while loops, and array shapes that depend on concrete JAX array values are allowed. Normally, this is disallowed when JIT-compiling a function and leads to a `ConcretizationTypeError`, because array shapes have to be statically known in compilation. For this reason, NumPyro, the JAX implementation of Pyro, also disallows these language features and only supports models with fixed and finite-dimensional structure.¹ In the following sections, we describe how we enable these language constructs with our custom JAX interpreter.

¹Note that Reichelt contributed a rudimentary implementation of DCC [59] and SDVI [46] to NumPyro which relies on user annotations for discrete branching variables.

$E ::=$	expression	$C ::=$	command
c	constant	skip	skip
x	variable	$x = E$	assignment
$f(E_1, \dots, E_n)$	function call	$C_1; C_2$	sequence
$S ::=$	string expression	if E then C_1 else C_2	if command
α	constant	while E do C	while loop
$g(S_1, \dots, S_m, E_1, \dots, E_n)$	function call	$x = \text{sample}(S, d(E_1, \dots, E_n))$	sample
		$\text{score}(S, d(E_1, \dots, E_n), E_{n+1})$	score
		$x = \text{concretize}(E)$	concretization

Fig. 3. Grammar of the core formal universal probabilistic programming language.

To give a precise technical description of our approach, we formalise the language following Lee et al. [28] and state the grammar in Figure 3. The formal PPL models the semantics of (1) by extending a standard while language with sample and score commands, corresponding to the absence or presence of an observed argument. We add a new concretize command which will be the only mechanism to convert arrays to scalar or strings values for use in branching conditions, addresses, and array constructors.

3.2 Density Semantics

In this section, we build on the density semantics of Lee et al. [28, Section 4.3]. For brevity, we focus on our core developments and refer to Appendix A for a complete review.

We introduce a type system $\mathbb{T} = \mathbb{G} \uplus \mathbb{A}$ with disjoint ground types $\mathbb{G} = \{\mathbb{B}, \mathbb{N}, \mathbb{Z}, \mathbb{R}\}$ and array types $\mathbb{A} = \{G[s_1, \dots, s_n] : n \geq 0, s_i \geq 1, G \in \mathbb{G}\}$, where we have $G[s_1, \dots, s_n] \cong G^{s_1, \dots, s_n}$. Let the set of all values be given by $\text{Val} = \uplus \mathbb{T} = \uplus_{T \in \mathbb{T}} T$. States are given by $\mathcal{S} = [\text{Var} \rightarrow \text{Val}]$ with reserved variables $\mathbf{p}, \mathbf{w}, \mathbf{b}$. Traces are typed finite maps from strings to arrays $\mathcal{T} = \uplus_{s \in \mathbb{S}} \prod_{(\alpha, A) \in s} A$, where $\mathbb{S}$ is the set of trace shapes – finite lexicographically ordered lists of pairs $(\alpha, A) \in \text{Str} \times \mathbb{A}$, similar to Lew et al. [29]. Both sets $\mathcal{S}$ and $\mathcal{T}$ are endowed with a σ -algebra and a reference measure. Commands C are interpreted as measurable functions $\llbracket C \rrbracket : \{\perp\} \uplus (\mathcal{S} \times \mathcal{T}) \rightarrow \{\perp\} \uplus (\mathcal{S} \times \mathcal{T})$:

$$\begin{aligned}
\llbracket \text{skip} \rrbracket(\sigma, \text{tr}) &= (\sigma, \text{tr}) \\
\llbracket x = E \rrbracket(\sigma, \text{tr}) &= \text{if } (\text{typeerr}(\sigma, E)) \text{ then } \perp \text{ else } (\sigma[x \mapsto \llbracket E \rrbracket \sigma], \text{tr}) \\
\llbracket C_1; C_2 \rrbracket(\sigma, \text{tr}) &= (\llbracket C_2 \rrbracket \circ \llbracket C_1 \rrbracket)(\sigma, \text{tr}) \\
\llbracket \text{if } E \text{ then } C_1 \text{ else } C_2 \rrbracket(\sigma, \text{tr}) &= \text{if } (\text{typeerr}(\sigma, E) \vee \llbracket E \rrbracket \sigma \notin \mathbb{B}) \text{ then } \perp \\
&\quad \text{else if } (\llbracket E \rrbracket \sigma = \text{true}) \text{ then } \llbracket C_1 \rrbracket(\sigma, \text{tr}) \text{ else } \llbracket C_2 \rrbracket(\sigma, \text{tr}) \\
\llbracket \text{while } E \text{ do } C \rrbracket(\sigma, \text{tr}) &= (\text{fix } G)(\sigma, \text{tr}), \\
&\quad \text{where } G(g)(\sigma, \text{tr}) = \text{if } (\text{typeerr}(\sigma, E) \vee \llbracket E \rrbracket \sigma \notin \mathbb{B}) \text{ then } \perp \\
&\quad \text{else if } (\llbracket E \rrbracket \sigma = \text{true}) \text{ then } (g \circ \llbracket C \rrbracket)(\sigma, \text{tr}) \text{ else } \llbracket \text{skip} \rrbracket(\sigma, \text{tr}) \\
\llbracket x = \text{sample}(S, d(E_1, \dots, E_n)) \rrbracket(\sigma, \text{tr}) &= \\
&\quad \text{if } (\text{typeerr}(\sigma, S, d(E_1, \dots, E_n)) \vee \llbracket S \rrbracket \sigma \notin \text{dom}(\text{tr}) \vee \text{tr}(\llbracket S \rrbracket \sigma) \notin \text{supp}(d)) \text{ then } \perp \\
&\quad \text{else } (\sigma[x \mapsto \text{tr}(\llbracket S \rrbracket \sigma), \mathbf{p} \mapsto \sigma(\mathbf{p}) \cdot \text{pdf}_d(\text{tr}(\llbracket S \rrbracket \sigma), \llbracket E_1 \rrbracket \sigma, \dots, \llbracket E_n \rrbracket \sigma)], \text{tr} \setminus \llbracket S \rrbracket \sigma) \\
\llbracket \text{score}(S, d(E_1, \dots, E_n), E_{n+1}) \rrbracket(\sigma, \text{tr}) &= \\
&\quad \text{if } (\text{typeerr}(\sigma, S, d(E_1, \dots, E_n), E_{n+1}) \vee \llbracket S \rrbracket \sigma \in \text{dom}(\text{tr}) \vee \llbracket E_{n+1} \rrbracket \sigma \notin \text{supp}(d)) \text{ then } \perp \\
&\quad \text{else } (\sigma[\mathbf{w} \mapsto \sigma(\mathbf{w}) \cdot \text{pdf}_d(\llbracket E_{n+1} \rrbracket \sigma, \llbracket E_1 \rrbracket \sigma, \dots, \llbracket E_n \rrbracket \sigma)], \text{tr})
\end{aligned}$$

The semantic functions for expressions are standard and $\llbracket C \rrbracket(\perp) = \perp$. Above we used $\beta \in \text{dom}(\text{tr})$ to indicate that $\exists i, B: s_i = (\beta, B)$, where s is the shape of the trace: $\text{tr} \in \prod_{(\alpha, A) \in \mathcal{S}} A$. In this case $\text{tr}(\beta) = \text{tr}_i$ and $\text{tr} \setminus \beta$ "deletes" β from tr . The semantics differ from Lee et al. [28] in three instances: 1) we introduced the dynamic type checks $\text{typeerr}(\sigma, E)$, which returns false for expression E if all function arguments have compatible type (plus similar checks for sample and score commands, see Appendix A.3); 2) instead of only Gaussians, we support multiple distributions d with density functions pdf_d , which requires the more general definition of traces and checking if values match the distribution type $\text{supp}(d) \in \mathbb{A}$; 3) we introduce the concretize command with

$$\llbracket x = \text{concretize}(E) \rrbracket(\sigma, \text{tr}) = \text{if } (\text{typeerr}(\sigma, E)) \text{ then } \perp \\ \text{else if } (\exists G \in \mathbb{G}: \llbracket E \rrbracket \sigma \in G[]) \text{ then } (\sigma[x \mapsto \iota(\llbracket E \rrbracket \sigma)], \text{tr}) \text{ else } \perp,$$

which takes a "0-dimensional array" $v \in G[] \cong G$ and converts it with the canonical isomorphism $\iota: G[] \rightarrow G$. To make this the only allowed way to compute a scalar or string from arrays, we restrict the types of functions: $\llbracket f \rrbracket \in \bigcup \{[(T_1 \times \dots \times T_n) \rightarrow A]: n \geq 0 \wedge T_i \in \mathbb{T} \wedge ((\exists i: T_i \in \mathbb{A}) \Rightarrow A \subseteq \biguplus \mathbb{A})\}$ and $\llbracket g \rrbracket \in \bigcup \{[(\text{Str}^m \times T_1 \times \dots \times T_n) \rightarrow \text{Str}]: m \geq 0 \wedge n \geq 0 \wedge T_i \in \mathbb{G}\}$.

Lastly, the density function for a probabilistic model defined by the command C is the measurable function $p_C: \mathcal{T} \rightarrow [0, \infty)$ given by

$$p_C(\text{tr}) = \begin{cases} \sigma(\mathbf{p}) \cdot \sigma(\mathbf{w}) & \text{if } \exists \sigma \in \mathcal{S}: \llbracket C \rrbracket(\sigma_0, \text{tr}) = (\sigma, \{\}), \\ 0 & \text{otherwise,} \end{cases}$$

where $\sigma_0 \in \mathcal{S}$ is some fixed initial state with $\sigma(\mathbf{p}) = 1$ and $\sigma(\mathbf{w}) = 1$.

3.3 SLP Program Transformation

Modelling the behavior of JAX interpreters, we introduce for each array type $A \in \mathbb{A}$ its *tracked dual* $A^\dagger \cong A$ to track which values depend on the input trace. We denote the isomorphisms with $\text{val}: A^\dagger \rightarrow A$ and $\cdot^\dagger: A \rightarrow A^\dagger$. We extend the types $\mathbb{T}^* = \mathbb{G} \uplus \mathbb{A} \uplus \mathbb{A}^\dagger$, $\text{Val}^* = \biguplus \mathbb{T}^*$, and states $\mathcal{S}^* = [\text{Var} \rightarrow \text{Val}^*]$. Tracked traces map strings to tracked arrays, $\mathcal{T}^\dagger = \biguplus_{s \in \mathcal{S}} \prod_{(\alpha, A) \in \mathcal{S}} A^\dagger$.

Our SLP generation process is comprised of a dynamic program analysis and a transformation, both formalised by modifying the semantics. The program analysis collects all concretized values that depend on the input trace in a list $\ell \in \mathcal{L}$, where $\mathcal{L} = \biguplus_{n \geq 0, G_i \in \mathbb{G}} \prod_{i=1}^n G_i$. This is formalised by $\llbracket \cdot \rrbracket^\dagger: \{\perp\} \uplus (\mathcal{S}^* \times \mathcal{T}^\dagger \times \mathcal{L}) \rightarrow \{\perp\} \uplus (\mathcal{S}^* \times \mathcal{T}^\dagger \times \mathcal{L})$ which is primarily new for concretize:

$$\llbracket x = \text{concretize}(E) \rrbracket^\dagger(\sigma, \text{tr}, \ell) = \text{if } (\text{typeerr}^\dagger(\sigma, E)) \text{ then } \perp \\ \text{else if } (\exists G \in \mathbb{G}: \llbracket E \rrbracket^\dagger \sigma \in G[]) \text{ then } (\sigma[x \mapsto \iota^\dagger(\llbracket E \rrbracket^\dagger \sigma)], \text{tr}, \ell.\text{append}(\iota^\dagger(\llbracket E \rrbracket^\dagger \sigma))) \quad (2) \\ \text{else if } (\exists G \in \mathbb{G}: \llbracket E \rrbracket^\dagger \sigma \in G[]) \text{ then } (\sigma[x \mapsto \iota(\llbracket E \rrbracket^\dagger \sigma)], \text{tr}, \ell) \text{ else } \perp,$$

where $\iota^\dagger = \iota \circ \text{val}: G[]^\dagger \rightarrow G$ and $\text{typeerr}^\dagger(\sigma, E)$ supports tracked types. For expressions, we define

$$\llbracket f(E_1 \dots, E_n) \rrbracket^\dagger \sigma = \begin{cases} \llbracket f \rrbracket(\text{val}^*(v_1), \dots, \text{val}^*(v_n))^\dagger & \text{if } (v_1, \dots, v_n) \in (\text{Val}^*)^n \setminus \text{Val}^n, \\ \llbracket f \rrbracket(v_1, \dots, v_n) & \text{otherwise,} \end{cases}$$

where $v_i = \llbracket E_i \rrbracket^\dagger \sigma \in \text{Val}^*$ and we defined $\text{val}^*(v) = (\text{if } (v \in \text{Val}) \text{ then } v \text{ else } \text{val}(v))$, such that the output of a function call is tracked if any of its arguments is tracked. Finally, we define the function $\text{decisions}_C: \mathcal{T} \rightarrow \{\perp\} \cup \mathcal{L}$ which returns the unique concretization decisions made by any trace:

$$\text{decisions}_C(\text{tr}) = \begin{cases} \ell & \text{if } \exists \sigma \in \mathcal{S}^*, \ell \in \mathcal{L}: \llbracket C \rrbracket^\dagger(\sigma_0, \text{tr}^\dagger, []) = (\sigma, \{\}, \ell), \\ \perp & \text{otherwise,} \end{cases}$$

where $\text{tr}^\dagger \in \mathcal{T}^\dagger$ is the tracked version of $\text{tr} \in \mathcal{T}$.

Now we are ready to define the program transformation parametrised by a list of decisions $\ell \in \mathcal{L}$ with "replay" semantics given by $\llbracket \cdot \rrbracket^\cup : \{\perp\} \uplus (\mathcal{S}^* \times \mathcal{T}^\dagger \times \mathcal{L}) \rightarrow \{\perp\} \uplus (\mathcal{S}^* \times \mathcal{T}^\dagger \times \mathcal{L})$. We have $\llbracket C \rrbracket^\cup = \llbracket C \rrbracket^\dagger$ except for the concretization command:

$$\begin{aligned} \llbracket x = \text{concretize}(E) \rrbracket^\cup(\sigma, \text{tr}, \ell) = & \text{if } (\text{typeerr}^\dagger(\sigma, E)) \text{ then } \perp \\ & \text{else if } (\exists G \in \mathbb{G}: \llbracket E \rrbracket^\dagger \sigma \in G[]^\dagger \wedge \ell \neq []) \text{ then} \\ & \quad (\sigma[x \mapsto \ell(1), \mathbf{b} \mapsto \sigma(\mathbf{b}) \wedge (\ell(1) = \iota^\dagger(\llbracket E \rrbracket^\dagger \sigma))], \text{tr}, \ell.\text{removefirst}()) \\ & \text{else if } (\exists G \in \mathbb{G}: \llbracket E \rrbracket^\dagger \sigma \in G[]) \text{ then } (\sigma[x \mapsto \iota(\llbracket E \rrbracket^\dagger \sigma)], \text{tr}, \ell) \text{ else } \perp \end{aligned} \quad (3)$$

Instead of using the concretized values of the input trace tr , $\llbracket \cdot \rrbracket^\cup$ consumes the pre-defined decisions ℓ and tracks mismatches at $\sigma(\mathbf{b}) \in \mathbb{B}$. The functions $p_C^\ell : \mathcal{T} \rightarrow [0, \infty)$ and $1_C^\ell : \mathcal{T} \rightarrow \mathbb{B}$

$$\begin{aligned} p_C^\ell(\text{tr}) &= \begin{cases} \sigma(\mathbf{p}) \cdot \sigma(\mathbf{w}) & \text{if } \exists \sigma \in \mathcal{S}^*: \llbracket C \rrbracket^\cup(\sigma_0[\mathbf{b} \mapsto 1], \text{tr}^\dagger, \ell) = (\sigma, \{\}, []), \\ 0 & \text{otherwise,} \end{cases} \\ 1_C^\ell(\text{tr}) &= \begin{cases} \sigma(\mathbf{b}) & \text{if } \exists \sigma \in \mathcal{S}^*: \llbracket C \rrbracket^\cup(\sigma_0[\mathbf{b} \mapsto 1], \text{tr}^\dagger, \ell) = (\sigma, \{\}, []), \\ 0 & \text{otherwise,} \end{cases} \end{aligned}$$

formally define a sub-model with following soundness properties (proven in Appendix B):

THEOREM 1. *Let C be a command and let $\text{dr} \in \mathcal{T}$ with shape $s \in \mathbb{S}$ be the decision representative of the sub-model with $\ell = \text{decisions}_C(\text{dr}) \neq \perp$. Then, p_C^ℓ and 1_C^ℓ are measurable functions. Further, there exists a command C^ℓ which is a sequence of assignment, sample and score commands,² such that*

$$\forall \text{tr} \in \mathcal{T}: 1_C^\ell(\text{tr}) = 1 \implies p_C^\ell(\text{tr}) = p_{C^\ell}(\text{tr}) = p_C(\text{tr}) \wedge \text{tr} \in \prod_{(\alpha, A) \in \mathcal{S}} A.$$

In words, if tr belongs to the same sub-model as dr , then p_C^ℓ computes the correct density, tr has the same shape as dr , and the sub-model can indeed be represented by a straight-line-program C^ℓ .

3.4 Compiling SLPs with JAX

By implementing the program analysis (2) and transformation (3) of the previous section as custom JAX interpreters, UPIX enables, for the first time, *automatic identification and compilation* of straight-line programs (SLPs) from a probabilistic program. Thus, without the need for annotations, we allow programs to contain control flow that depends on random variables and allow programs to contain distribution objects whose support structure depends on other random variables.

Under the hood, this automation is driven by JAX's ability to compile a pure, array-manipulating Python function by executing it on input arrays wrapped in so-called *tracer* objects. Each JAX interpreter implements the behavior of operations on these tracers, records every operation performed on them, and builds an intermediate representation called JAXPR. JAX can then hand off the computation to XLA for hardware-specific optimisation on CPUs, GPUs, or TPUs.

To compile SLPs, we first use the decorator `@model` to transform a function `def f(args: S) -> T` to `model(f): S -> (Dict[str, Array] -> (Float, T))` by adding the trace `tr: Dict[str, Array]` as input argument and by replacing sample statements with the density semantics:

```
value = observed if observed is not None else tr[address]
lp += distribution.log_prob(value)
return value
```

That is, `model(f)(args)(tr)` runs the program with respect to input trace `tr` and outputs its probability density alongside the return value. The arguments `args` serve as model hyper-parameters.

²In addition to control flow, we may pre-define all array shapes in the program with ℓ by restricting array constructors to only work for concrete scalar arguments, e.g., $\llbracket \text{zeros}_{\mathbb{R}} \rrbracket : \mathbb{N} \rightarrow \biguplus_{n \in \mathbb{N}} \mathbb{R}[n]$ initialises an array with 0s for a given length.

The JAX tracer class makes it possible to detect and intercept concretization operations. The interpreter corresponding to the program analysis (2) circumvents the `ConcretizationTypeError` and collects the concrete values in a decision list. The interpreter corresponding to the program transformation (3) consumes this decision list and transforms `model(f)(args)` to use these pre-defined concrete values at the concretization operations instead of the values computed by the input trace. In addition, it tracks the mismatch of pre-defined decisions and decisions computed from the input trace returning the result in addition to the density.

In Figure 4, we illustrate this process by giving two example SLP generations for a model which exhibits stochastic support. Note that the branching condition, the set of executed sample statements, and the dimensionality of the random variables with addresses `f"{x_i}"` depend on the input trace. Unlike existing implementations of DCC, our program transformation automatically compiles these SLPs without the need to annotate that the support structure depends on `U` and `N`.

```

@model
def simple():
    U = sample("U", Uniform(0,1))
    if U > 0.5:
        N = 1
    else:
        N = sample("N", Poisson(3))
    i = 0
    while i < N:
        sample(f"x_{i}",
              Normal(zeros((N,)), 1,))
        i = i + 1

def SLP1(tr: Dict[str, Array]):
    lp = 0
    lp += Uniform(0,1).log_prob(tr["U"])
    lp += Normal([0],1).log_prob(tr["x_0"])
    return lp, (tr["U"] > 0.5)

def SLP2(tr: Dict[str, Array]):
    lp = 0
    lp += Uniform(0,1).log_prob(tr["U"])
    lp += Poisson(3).log_prob(tr["N"])
    lp += Normal([0,0,0],1).log_prob(tr["x_0"])
    lp += Normal([0,0,0],1).log_prob(tr["x_1"])
    lp += Normal([0,0,0],1).log_prob(tr["x_2"])
    return lp, (tr["U"] <= 0.5) and (tr["N"] == 3)

```

Fig. 4. SLP generation process exemplified: Left: a probabilistic model defined in `UPIX` that exhibits stochastic support. Right: Two example straight-line programs which are equivalent to programs obtained with the JAX transformations described in Section 3.4. As typical, densities are computed in log-space.

Note that the described JAX transformations are implemented to be compatible with other JAX transformations. This means that, for instance, we can use `jax.vmap` to vectorise the log-probability density computation or `jax.grad` to differentiate the log-probability density with respect to the trace. This enables an efficient implementation of gradient-based inference algorithms like HMC [6] or ADVI [26]. Lastly, the transformations are hidden from the user in the `SLP` class, which provides methods for evaluating the density, generating samples from the prior, etc.

3.5 Abstract Divide-Conquer-Combine

At the highest level, we developed a novel abstraction of the DCC approach shown in Listing 1. The `initialise_active_slps` and `update_active_slps` methods implement the divide step. The former finds SLPs (sub-models) in the initialisation phase and in the update phase the latter retains promising SLPs while discarding less promising SLPs based on the results collected so far. New SLPs may also be instantiated and made active in the update phase. The `run_inference` method implements the conquer step and performs inference for all active SLPs and stores the result. The `estimate_log_weight` is responsible for computing the contribution weight of the SLP density to the full model density. Together with the `combine` method the combine step of DCC is implemented. In this last step, all results are aggregated with respect to the estimated contribution weights. In Section 4, we will instantiate this abstract routine based on several Bayesian inference approaches to implement existing DCC methods and to formulate novel DCC algorithms.

```

442 1 class DCC:
443 2     def run(self, model):
444 3         active_slps, inactive_slps, results = [], [], []
445 4         self.initialise_active_slps(model, active_slps, inactive_slps)
446 5         while len(active_slps) > 0:
447 6             for slp in active_slps:
448 7                 self.run_inference(slp, results)
449 8                 self.estimate_log_weight(slp, results)
450 9                 self.update_active_slps(model, results, active_slps, inactive_slps)
451 10            return self.combine(results)

```

Listing 1. The divide-conquer-combine approach as abstracted in UPIX.

3.6 Programmable Inference

In principle, every step of DCC is customisable in UPIX. We provide several instantiations of the framework for the most popular inference algorithms including Markov chain Monte Carlo (MCMC), sequential Monte Carlo (SMC), and Variational Inference (VI) algorithms. These instantiations are organised hierarchically as shown in Figure 5 and provide default implementations of the individual steps of DCC as well as convenience constructors to ease programmable inference based on established principles [7, 13, 30, 42]. In Section 4, we will describe these instantiations in detail.

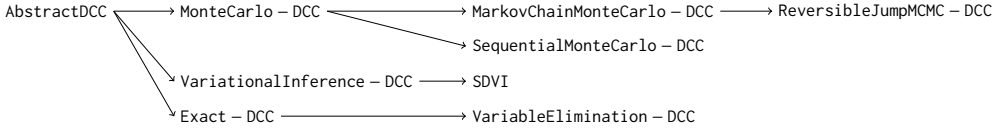


Fig. 5. Hierarchical organisation of the instantiations of DCC algorithms in UPIX.

Unlike existing universal probabilistic programming systems which allow the user to customise inference only for the full model, UPIX enables the user to customise inference for each sub-model encoded by an SLP. Crucially, at the SLP level, the trace data structure `tr: Dict[str, Array]` has fixed array shapes which allows the compilation of its (log-)probability density function. This enables us to also compile and execute full inference routines programmed for each SLP with the provided convenience constructors and run them on CPUs, GPUs, or TPUs.

Furthermore, note that at the SLP level, the log-probability density function can be "flattened" to the form `log_p: Array -> Float` by concatenating the values of the input trace in fixed order. Most of the inference algorithms implemented in UPIX, which form the building blocks of the programmable inference constructs, are implemented based solely on a JAX-compiled log-probability density function. This opens up opportunities for interoperability with existing inference systems. For instance, BlackJAX [8] is a Bayesian inference framework built around the log-probability density. But also parts of the inference machinery of TensorflowProbability [27] or NumPyro [45] could be integrated in UPIX as new building blocks for programmable inference.

Lastly, while we provide default implementations for the `initialise_active_slps` method and for the `update_active_slps` method, it is generally beneficial to adapt them to the model at hand. We give examples in Section 4. Effectively, this gives rise to a two-layer approach to programmable inference. First, inference routines may be customised at the SLP level in the conquer step. Second, on the meta-inference level, users may customise how the space of SLPs is explored and how computational resources are allocated to them.

3.7 Parallelisation

The DCC approach together with JAX compilation allow for many opportunities to parallelise inference across computational devices.

First, UPIX implements vectorisation techniques for the inference run and the estimation of the weight of each SLP. In this parallelisation approach, computations are scheduled sequentially in lines 6-8 of Listing 1.

Vectorisation. The vectorisation transform of JAX called `jax.vmap` was used to implement the inference machinery as numerical computations on multi-dimensional arrays. Such computations can be efficiently run on accelerator hardware designed for single-instruction multiple-data (SIMD) workloads like GPUs and TPUs. This parallelisation technique is recommended for machines with a single GPU.

Sharded Arrays. On top of vectorisation, JAX allows the distribution of computations on multi-dimensional arrays among several accelerators with `jax.pmap` and `jax.shard_map`. Thus, we can run inference and weight estimation for each SLP on multiple GPUs and TPUs. This is recommended for inference and weight estimation workloads which are embarrassingly parallel. In Section 5, we use this technique to scale MCMC to thousands of chains, VI to thousands of gradient estimates per step, and SMC to thousands of particles.

Second, UPIX implements parallelisation techniques to run inference and weight estimation for multiple SLPs at the same time, i.e., computations in lines 6-8 of Listing 1 are scheduled in parallel.

Multiple accelerator devices. If the inference and weight estimation workloads of single SLPs are not embarrassingly parallel, we may distribute the full workloads among GPUs and TPUs which work independently in a multi-processing fashion.

Multiple CPU cores. By default, JAX does not allow dispatching multiple computations to specific CPU cores in parallel. As some users of UPIX may not have access to machines with multiple accelerators, we implemented a workaround to run inference and weight estimation of multiple SLPs with multi-processing. To achieve this, we launch multiple JAX processes pinned to single CPU cores and send them workloads via the serialisation functionality built into JAX.

4 DCC Algorithms Instantiated with UPIX

In five case studies, this section demonstrates UPIX's ability to efficiently implement existing and novel DCC algorithms. First and foremost, we evaluate inference throughput by putting the runtime of UPIX in context of existing solutions. Whenever possible, we also quantify approximation quality. However, our goal is not to establish these algorithms, or DCC as a whole, as the leading benchmarks for these models, but to show how UPIX paves the way for entirely new classes of DCC algorithms.

4.1 Markov Chain Monte Carlo DCC

As the first application of UPIX, we adapt the DCC algorithm of Zhou et al. [59], which is built around MCMC inference. MCMC algorithms build a chain of samples x_i which are perturbed by a kernel $x_{i+1} \sim k(\cdot|x_i)$ that leaves the target distribution invariant. This property implies that with a sufficiently long chain, a sample from the target distribution is approximated. Such kernels can be constructed even if the target density function is only known up to a multiplicative constant, e.g., the joint probability density over latent and observed variables can be used to approximate the posterior distribution. Notable examples are the Metropolis-Hastings [39], the HMC [6], or the NUTS kernel [22].

The MCMC DCC algorithm is implemented in UPIX as sketched in Listing 2. It is realised as a sub-class of `MonteCarloDCC` our abstract implementation of DCC which approximates the posterior with a weighted set of traces. The initialisation of active SLPs is done by sampling a trace from the model prior and checking if the trace is not in the support of any SLP found so far. As proposed by Zhou et al. [59], active SLPs are updated by running a *global* random walk Metropolis-Hastings algorithm on the full model, where traces may change with respect to their set of addresses and array shapes. For this reason, this global algorithm cannot be JIT-compiled and serves solely for updating the set of active SLPs. In the `MarkovChainMonteCarloDCC` class the normalisation constant of an SLP is estimated via importance sampling as in Zhou et al. [59]. To ease programmable inference for MCMC-based DCC, we provide convenience constructs adapted from prior work [13, 15] to define MCMC kernels for each SLP in the `get_MCMC_inference_regime` method, which are then used in the MCMC routine launched in the `run_inference` method.

```
class MonteCarloDCC(AbstractDCC):
    def initialise_active_slps(self, model, active_slps, inactive_slps):
        # draw traces from prior
    def update_active_slps(self, model, results, active_slps, inactive_slps):
        # global random walk updates
    def combine(self, results):
        # combine inference results as set of weighted traces
class MarkovChainMonteCarloDCC(MonteCarloDCC):
    def estimate_log_weight(self, slp, results):
        # estimate log-weight with importance sampling
    def get_MCMC_inference_regime(self, slp):
        # provided by the user
    def run_inference(self, slp, results):
        # run MCMC routine based on self.get_MCMC_inference_regime(slp)
```

Listing 2. Implementation sketch of Markov Chain Monte Carlo DCC in UPIX.

Our programmable inference approach to MCMC-based DCC is best illustrated with an example. Mak et al. [33] introduced the Pedestrian model shown in Figure 6 to motivate NP-DHMC – their novel non-parametric variant of discontinuous HMC [43] that can handle both discontinuities in the density function and stochastic support. The Pedestrian model is challenging, because the while loop condition and the observation depend on all random variables sampled so far. In the spirit of DCC, we approximate the posterior of this model by running discontinuous HMC individually for each SLP and combine the results later. UPIX automatically splits up this program into SLP_k , $k \in \mathbb{N} \setminus \{0\}$, where SLP_k corresponds to the program that executes the while loop for k iterations.

```
@model
def pedestrian():
    position = sample("start", Uniform(0,3))
    distance, t = (0.0, 0)
    while (position > 0) & (distance < 10):
        t += 1
        step = sample(f"step_{t}", Uniform(-1,1))
        position += step
        distance += jax.lax.abs(step)
    sample("obs", Normal(distance, 0.1), observed=1.1)

regime = MCMCSteps(
    MCMCStep(
        Variables("start"),
        MH(lambda _: Uniform(0,3))
    ),
    MCMCStep(
        Variables(r"step_{d+}",
        DHMC(50, 0.05, 0.15)
    )
)
```

Fig. 6. Left: Pedestrian model [33] in UPIX. Right: MCMC routine specified with provided constructs.

On the right in Figure 6, we show how users may customise MCMC kernels for the Pedestrian model with the provided constructs. They can choose from a set of inference algorithms like Metropolis-Hastings `MH` or discontinuous HMC `DHMC`, customise the proposal distributions and hyper-parameters, and apply them to a subset of variables specified with regular expressions.

To compare our approach, we ran the NP-DHMC PyTorch implementation of Mak et al. [33] with 8 chains of 1 000 samples configured with their best scoring hyper-parameter values: 50 HMC steps with step-size 0.1. Running each chain in multiple processes in parallel takes about 160 seconds on a 10 core M2 Pro machine (averaged over 5 runs). In less than 25 seconds (our reported runtimes always include JAX compile time), we can run 8 chains of 25 000 samples in `UPix-MCMC-DCC` with the same configuration for DHMC. This runs inference and combines results for SLP_1 to SLP_6 . It was determined automatically that all other SLPs do not contribute enough to the posterior to warrant an inference run. For this, we customised `initialise_active_slps` to iterate over the SLPs ordered by number of steps and stop adding SLP_k to `active_slps` once the normalisation constant of SLP_k is estimated to be a factor of 10^3 smaller than the highest weighted SLP.

Thus, with inference routines JIT-compiled for each SLP we are able to take 960 times more samples per second than the NP-DHMC algorithm which has to move between SLPs in a non-compiled program. In Figure 7 we show the qualitative difference between the achieved approximations to the true posterior. The approximation quality is quantified with $L_\infty(\hat{F}, F) = \max_x |\hat{F}(x) - F(x)|$, where F is the ground truth posterior cumulative distribution function over the "start" variable x , which is highly dependent on all other variables, and $\hat{F}$ is the approximation obtained with the MCMC approaches. The ground truth is estimated with 10^{12} importance samples. NP-DHMC achieves $L_\infty \approx 0.0274$ while our `UPix` implementation achieves $L_\infty \approx 0.0016$.

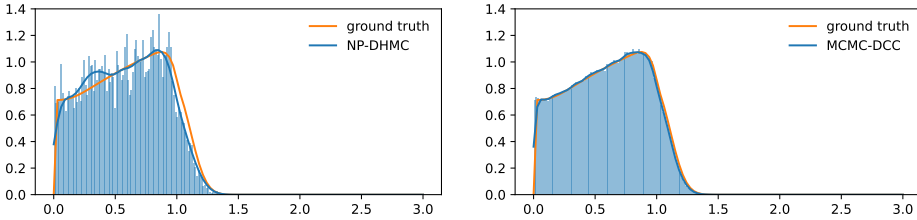


Fig. 7. Approximations to the true posterior of the Pedestrian model (estimated with 10^{12} importance samples) with $8 \cdot 1\,000$ samples from NP-DHMC versus $6 \cdot 8 \cdot 25\,000$ samples from MCMC-DCC as 100 bin histograms.

4.2 Variational Inference DCC: SDVI

In Variational Inference the posterior distribution is approximated by optimising the parameters ϕ of a *variational distribution* q_ϕ to minimise the Kullback–Leibler divergence to the target p . In practice, the so-called *evidence-lower-bound* (ELBO) $\mathcal{L}(\phi) = \mathbb{E}_{z \sim q_\phi} [\log p(z) - \log q_\phi(z)]$ is maximised which constitutes an equivalent but more practical optimisation objective. This optimisation is achieved via gradient ascent and is typically implemented on top of automatic differentiation machinery in probabilistic programming systems. In this case, the approach is called *Automatic Differentiation Variational Inference* (ADVI) [26].

Reichelt et al. [46] proved that when we fit a local variational distribution $q_{\phi_k}^k$ to the posterior of each individual sub-model encoded by an SLP, then the mixture of the local variational distributions, where each $q_{\phi_k}^k$ is weighted proportional to their local ELBO, $w_k \propto \exp(\mathcal{L}_k(\phi_k))$, is the optimal approximation to the full model. This enabled their variational inference based DCC approach.

This approach is implemented in UPIX as sketched in Listing 3. To ease programmable inference we allow the user to specify a variational distribution as a so-called *guide program* following Pyro’s approach [7]. A guide program in UPIX is a Python function made up from sample statements `sample(...)` and parameter declarations `param(...)`. The distributions in sample statements are parameterised with respect to the declared parameters and we can compute the ELBO gradient via JAX’s automated differentiation machinery. In addition, we also make auto-guides available to the user that approximate the posterior with a parameterised multivariate Gaussian distribution.

```
class VariationalInferenceDCC(AbstractDCC):
    def initialise_active_slps(self, model, active_slps, inactive_slps):
        # draw traces from prior
    def get_guide(self, slp):
        # provided by the user
    def run_inference(self, slp, results):
        # run ADVI routine based on self.get_guide(slp)
    def estimate_log_weight(self, slp, results):
        # estimate log-weight with local ELBO
    def combine(self, results):
        # combine inference results as a mixture of variational distributions
class SDVI(VariationalInferenceDCC):
    def update_active_slps(self, model, results, active_slps, inactive_slps):
        # use SuccessiveHalving strategy by Reichelt et al.
```

Listing 3. Implementation sketch of Variational Inference DCC and SDVI in UPIX.

Reichelt et al. [46] also proposed a *Successive Halving* strategy to allocate computational resources to the most promising SLPs. In this strategy, a fixed computational budget is divided among SLPs for a number of phases. After each phase, half of the SLPs are discarded, while the other half is optimised with double the number of ADVI iterations. Their full method is called *Support Decomposition Variational Inference* (SDVI) which we also implemented in UPIX via the `update_active_slps` method, see Listing 3.

We compare the runtime of their Pyro implementation of SDVI against our implementation in UPIX. As a benchmark, we implement a probabilistic program to infer the kernel structure of a Gaussian Process model for airline passenger data. The kernel structure is given by the probabilistic context-free grammar $K ::= \text{SE}|\text{RQ}|\text{PER}|\text{LIN}|K + K|K \times K$, where we consider the squared-exponential, rational-quadratic, periodic, and linear primitive kernels. This model exhibits stochastic support as the number of combined base kernels is not fixed and unbounded. We have a unique SLP for each possible kernel structure. The prior over the kernel grammar is implemented as a recursive Python function. Like Reichel et al., we also use an auto-guide for each SLP.

The Pyro version with 10 parallel processes takes 18 minutes and 45 seconds on a M2 Pro to run ADVI on initially about 110 random SLPs reduced to the 10 best SLPs over 5 phases with a budget of 10^6 total ADVI steps. Our equivalent UPIX implementation takes only 2 minutes and 41 seconds. Here, we use the *Multiple CPU cores* parallelisation strategy with 10 parallel JAX processes, see Section 3.7. As computing a ground truth posterior for the Gaussian process model is infeasible, we cannot compare the approximation quality of both approaches directly. Nevertheless, we report the model-comparison-metric *log-posterior predictive density* (LPD) on a 10% hold-out test set. It is given by $\text{LPD} = \log \int_{\mathcal{T}} p(\mathcal{D}_{\text{test}}|\text{tr}, \mathcal{D}_{\text{train}}) \hat{p}(\text{tr}|\mathcal{D}_{\text{train}}) d\text{tr}$, where we made the conditioning on data sets explicit. The metric measures how well the approximation $\hat{p}$, based on training data, can predict unseen test data. Our implementation achieves 6.90 ± 0.38 while the LPD of the reference SDVI implementation is slightly worse with -5.58 ± 0.19 .

4.3 Reversible Jump DCC

Reversible Jump MCMC (RJMCMC) algorithms [19], which belong to the class of *involutive* MCMC algorithms [42], were developed for models that can be decomposed into enumerable sub-models of varying dimension. Metropolis-Hastings moves are designed to switch between these sub-models during inference. As suggested by the name, these jumps have to be designed in a reversible manner: for each jump move there has to be a move which reverses it, establishing a bijection. In this section, we describe our novel DCC algorithm based on this principle.

This approach is best explained by example. In one of the earliest formulations Richardson and Green [48] designed reversible jumps for a Gaussian mixture model (GMM) with unknown number of components. Mathematically, we model the number of components $K \sim \text{Poisson}(2) + 1$ and the membership probability as $w \sim \text{Dirichlet}(\delta)$. Data is modelled with $z_i \sim \text{Categorical}(w)$ and $x_i \sim \text{Normal}(\mu_{z_i}, \sigma_{z_i}^2)$, where $\mu_k \sim \text{Normal}(0, \kappa^{-2})$ and $\sigma_k \sim \text{InverseGamma}(\alpha, \beta)$ are the center and deviation around the center for component k . We have implemented this model in UPIX as shown in Figure 2. For this model, the jumps are given by 1) splitting one component into two components and 2) merging two components to one component. These jumps are designed to be inverses of each other on an extended variable space. The RJMCMC inference algorithm for the GMM alternates between the reversible jumps, changing the number of components, and well-known Gibbs moves to draw samples from the posterior of each sub-model. These Gibbs moves are readily implemented with the MCMC programmable inference constructs provided by UPIX.

At first, we implemented inference for the GMM following the MCMC-DCC approach as described in Section 4.1 which eliminates the need to jump between sub-models as every SLP corresponds to a sub-model with a certain fixed number of components. However, we were not able to accurately estimate the normalisation constants needed to combine the results with importance sampling methods as proposed by Zhou et al. [59] or with other common approaches for marginal likelihood estimation like Chib's method or sequential Monte Carlo methods [31].

Subsequently, we realised that for the combine step to be sound in DCC, the absolute normalisation constants of each SLP are not required. Instead, it is sufficient to weigh an SLP only *relative* to the other SLPs. Even more, this relative estimation can be built around reversible jumps. To see this, let $\mathcal{D}$ be the observed data and let $p(\cdot|i, \mathcal{D})$ denote the finite-dimensional posterior of sub-model i and $k(j, y|i, x)$ a reversible jump kernel that moves from sub-model i to sub-model j . We observe following relationship between normalisation constants $p(i|\mathcal{D})$ and $p(j|\mathcal{D})$:

$$\begin{aligned} Z_{j \leftarrow i} &:= \int_{\mathcal{T}_j} \int_{\mathcal{T}_i} k(j, y|i, x) p(x|i, \mathcal{D}) dx dy = \frac{1}{p(i|\mathcal{D})} \int_{\mathcal{T}_j} \int_{\mathcal{T}_i} k(j, y|i, x) p(i, x|\mathcal{D}) dx dy \\ &= \frac{1}{p(i|\mathcal{D})} \int_{\mathcal{T}_i} \int_{\mathcal{T}_j} k(i, x|j, y) p(j, y|\mathcal{D}) dy dx = \frac{p(j|\mathcal{D})}{p(i|\mathcal{D})} Z_{i \leftarrow j} \end{aligned}$$

Above, the second to last equation follows from the fact that involutive MCMC kernels such as RJMCMC kernels satisfy *detailed balance* [42]. This gives rise to an infinite system of equations

$$Z_{j \leftarrow i} \cdot p(i|\mathcal{D}) = Z_{i \leftarrow j} \cdot p(j|\mathcal{D}), \quad i \in \mathbb{N}, j \in \mathbb{N}, \quad (4)$$

which can be viewed as a reversible discrete Markov chain with stationary distribution $\pi_i = p(i|\mathcal{D})$. This system can be approximated and solved via truncation [16], i.e., only taking into account the finite number of SLPs for which inference was performed in DCC. The quantities $Z_{j \leftarrow i}$ may be approximated through sampling as follows: First, take $x \sim p(\cdot|i, \mathcal{D})$, where the posterior $p(\cdot|i, \mathcal{D})$ is approximated by running inference for SLP_{*i*}. Then, we simulate reversible jumps $(j, y) \sim k(\cdot|i, x)$ and take $\hat{Z}_{j \leftarrow i}$ as the empirical marginal distribution over j . Crucially, this estimation can often be JIT-compiled as both SLP_{*i*} and SLP_{*j*} have static support structure. Note that while this gives

reasonable justification, a formal proof that this truncation and coefficient approximation produces consistent estimates is beyond the scope of this work.

To ease programmable inference, a language for declaring reversible jumps and computing acceptance rates via automatic differentiation was implemented following Cusumano-Towner et al. [12]. The `estimate_log_weight` method builds the system of equation (4) based on the user-provided reversible jumps and `combine` solves it numerically to estimate the SLP weights.

To evaluate our approach, we compare against the described RJMCMC algorithm for the GMM implemented in Gen by Matheos et al. [38]. Running 8 MCMC chains of length 25 000 in a multi-threaded fashion takes 94 seconds (with ≈ 13 seconds Julia JIT-compile time). This results in an approximation to the posterior with error $L_\infty(\hat{F}, F) = \max_K |\hat{F}(K) - F(K)| \approx 0.0045$, where F is the ground truth cumulative distribution function over the number of components K estimated by collecting a total of 10^7 samples and $\hat{F}$ its approximation. Note that to correctly estimate the distribution of K , all sub-models have to be weighted accurately. Our approach takes 92 seconds (≈ 47 seconds compile-time) to run 8 chains and 25 000 samples for 11 SLPs corresponding to component numbers 1 to 11 (with Gen the highest number was 9). For this, we customised `update_active_slps` to run inference on SLPs ordered by the component count and SLP_{k+1} is made active only if the split probability of SLP_k exceeds the threshold 0.01. In approximately the same time, UPIX produces 10 times more samples compared to Gen, which results in a similar estimate $L_\infty(\hat{F}, F) \approx 0.0042$.

4.4 Sequential Monte Carlo DCC

Although the sequential Monte Carlo (SMC) [11] inference algorithm class can be quite naturally instantiated as a DCC algorithm, to the best of our knowledge, we are the first to do so. Different to MCMC and VI, SMC algorithms propagate a set of samples – so-called *particles* – once through the probabilistic program. Observed data is added to the inference process iteratively. After each addition of data, the particles are re-weighted relative to their likelihood. Then, particles may be resampled according to their weights effectively discarding low-probability particles and duplicating high-probability particles. Optionally, a *rejuvenation* MCMC kernel may be applied to all particles after resampling to make them more diverse. At the end, the set of weighted particles constitutes an approximation to the posterior distribution of the model. As a convenient by-product SMC algorithms also provide an estimate of the marginal likelihood Z .

Thus, an SMC-DCC algorithm is implemented in UPIX as sketched in Listing 4, where we reuse the marginal likelihood estimate from the inference run in the combine step. We enable the user to easily specify the schedule with which data is introduced to the inference process in the `get_SMC_data_schedule` method. For instance, this could be one data point at a time or a batch of data points at a time. To run SMC efficiently, many probabilistic programming systems implement machinery to incrementally execute the program and add data this way. A similar approach in UPIX would lead to a lot of compilation overhead as the log-probability density function would have to be compiled for every data increment. Instead, UPIX always execute the full probabilistic program, but passes a mask as argument that determines which data points contribute to the log-probability density. This way the density computation has to be compiled only once per SLP in SMC. In addition to making the data schedule programmable, we also provide the option to specify a rejuvenation MCMC kernel with the same constructs as in Section 4.1. We also provide many options to customise the resampling scheme in SMC like the resampling method (multinomial, stratified, and systematic) and resampling time (before or after rejuvenation). Note that while the presented SMC formulation is flexible, it represents just one instance of the broader SMC framework, which accommodates arbitrary density sequences and any proposal mechanism that preserves proper weighting.

```

785 class SequentialMonteCarloDCC(MonteCarloDCC):
786     def get_SMC_data_schedule(self, slp):
787         # provided by the user, may be customised for each slp
788     def get_SMC_rejuvenation_regime(self, slp):
789         # provided by the user, may be customised for each slp
790     def run_inference(self, slp, results):
791         # run SMC routine based on rejuvenation kernel and data schedule
792     def estimate_log_weight(self, slp, results):
793         # estimate log-weight with estimate from SMC inference run

```

Listing 4. Implementation sketch of Sequential Monte Carlo DCC in UPIX.

To test this approach, we again consider the Gaussian Process model from Section 4.2, but with slightly different kernel grammar $K ::= GE|PER|LIN|K + K|K \times K$, where GE is a gamma-exponential primitive kernel. At the start we make the three primitive kernels active SLPs. On each of four subsequent DCC iterations, we run inference for five active SLPs which are obtained by updating the previous set of active SLPs with global updates that randomly perturb the GP kernels. We use 100 SMC particles, a HMC rejuvenation kernel, and we split the airline passenger data set in 10 sub-sets. On a M2 Pro this takes around 5 minutes and 25 seconds to complete.

This example is based on Saad et al. [50], who designed an SMC approach with involutive MCMC rejuvenation moves for time series structure discovery and implemented it in the Julia package AutoGP.jl. We configured our UPIX-SMC-DCC instantiation to be as close to their implementation as possible with the difference that we do not use involutive MCMC rejuvenation moves as our approach does not move between sub-models during inference. On the same hardware, AutoGP.jl with 100 particles takes around 8 minutes and 50 seconds to finish. In this time, the 100 particles are propagated only once through the full model, changing the structure of the Gaussian process kernel many times. In contrast, our approach propagates 100 particles through a total of 23 SLPs (selected from 100 SLPs proposed by the global updates), keeping the structure fixed. The former has the advantage of exploring more kernel structures, while the latter performs inference and marginal likelihood estimation more accurately for a smaller set of kernel structures.

This comparison serves primarily to put the runtime of our approach into perspective. Evaluating the approximation quality of both approaches is difficult, again, due to the infeasibility of computing a ground truth. The model-comparison metric LPD is similar for both methods, see Table 1.

4.5 Variable Elimination DCC

In the conquer step of DCC, we are not limited to approximate inference algorithms which we have considered up until this point. In this section, we present a novel instantiation of the DCC framework which performs inference for discrete models with stochastic support by exactly solving the sub-models which have static structure.

It is well known that discrete probabilistic models with a fixed finite number of random variables can be solved exactly [25]. This can be achieved by naively enumerating the model support and computing the marginal likelihood. However, as this becomes infeasible for larger models, more sophisticated methods like *Variable Elimination* (VE) or Belief Propagation have to be used [25]. In variable elimination, the probability density function of a model is given in factorised form $p(X) = \prod_{k=1}^K f_k(X_k)$, where $X_k \subseteq X$ are subsets of variables. Typically, the factors have the form of conditional probability tables, i.e. $p(x|\text{parents}(x))$. This representation allows the efficient marginalisation of a variable x by building the factor product of all factors to which the variable x belongs, and then summing out x . This new factor $f'(X) = \sum_x \prod_{k: x \in X_k} f_k(X_k)$ replaces all the factors used in the product and gives rise to a new factorisation without x . This elimination process

Table 1. Comparison of DCC algorithms implemented in UPIX modelled after baseline algorithms and executed on consumer hardware (M2 Pro CPU). Averaged over 5 runs.

Model	Method	Details	Runtime	$L_\infty \downarrow$	LPD $\uparrow$
Pedestrian	NP-DHMC [33]	$8 \cdot 1\,000$ samples	$162 \pm 63s$	0.0274 ± 0.00418	-
	UPIX MCMC-DCC	$6 \cdot 8 \cdot 25\,000$ samples	$23 \pm 0.3s$	0.0016 ± 0.00047	-
GP	SDVI [46]	10^6 step budget	$1126 \pm 105s$	-	-5.58 ± 0.19
	UPIX VI-DCC	10^6 step budget	$161 \pm 4s$	-	6.90 ± 0.38
GMM	Gen [38]	$8 \cdot 25\,000$ samples	$94 \pm 1.3s$	0.0045 ± 0.0018	-
	UPIX RJ-DCC	$11 \cdot 8 \cdot 25\,000$ samples	$92 \pm 0.9s$	0.0042 ± 0.0012	-
GP	AutoGP [50]	100 particles	$531 \pm 87s$	-	31.27 ± 0.47
	UPIX SMC-DCC	100 particles, 23 SLPs	$324 \pm 18s$	-	33.65 ± 0.68
Urn	Dice [23]	truncated $N \leq 19$	$2504 \pm 86s$	$5.8 \cdot 10^{-6} \pm 0.0$	-
	UPIX VE-DCC	solved SLPs for $N \leq 20$	$12.6 \pm 0.1s$	$1.3 \cdot 10^{-6} \pm 0.0$	-

can be repeated until only a set of *query* variables remains or until all variables are eliminated such that the result of the computation is the normalisation constant.

We have implemented a DCC version of variable elimination in UPIX as follows. In the method `get_query_variables`, the user may specify query variables. As the efficiency of variable elimination heavily depends on the elimination order, we enable the user to customise it with a `get_elimination_order` method, but we also provide a strong default ordering. We have implemented a JAX interpreter which automatically constructs the conditional probability table for each variable in the sub-model, thus setting up the factorisation. By default, this is done one factor at a time, which may cause a lot of compilation overhead. To remedy this, we give the user the option to specify sets of variables for which the factors can be computed in one pass in a `get_factors` method. Finally, variable elimination is performed in `run_inference` for each SLP and we can use the exact normalisation constant computed as a by-product as weight in `estimate_log_weight`.

To test the variable elimination DCC approach, we consider the Urn model of Milch et al. [40]. In this model, we have an urn with an unknown number of balls $N \sim \text{Poisson}(6)$ each of which has equal prior probability of being black or white. We repeatedly draw a ball, observe its color with 80% accuracy, and put it back. With posterior inference we aim to answer: Given that we observed 5 white and 5 black balls, how many balls are inside the urn? As the number of balls is unknown, the support structure of this model is stochastic and cannot be directly solved with exact inference. With our UPIX implementation it takes 13 seconds to compute the factorisation and the exact solution for 1 to 20 balls (64 seconds with unoptimised factor computation). In this implementation, we customised SLP initialisation to be performed in order of the number of balls. This gives a maximum absolute error of $1.315 \cdot 10^{-6}$ to the true posterior probabilities mass function of N , computed analytically. For reference, solving the truncated model with $N \leq 19$ in Dice [23] takes over 40 minutes. Truncating at $N \leq 20$ exceeds the 32 GB memory limit. To the best of our knowledge, there exists no exact inference algorithm that can solve the full model.

5 Scaling Experiments

So far, we have demonstrated the capabilities of UPIX only on consumer-grade hardware like the M2 pro CPU. In comparison to existing probabilistic inference implementations, we observed that UPIX enables faster inference which can result in better approximations as summarised in Table 1. In this section, we shift our attention to large-scale computational hardware and evaluate the capabilities of UPIX on machines with up to 64 CPU cores and 8 GPU devices.

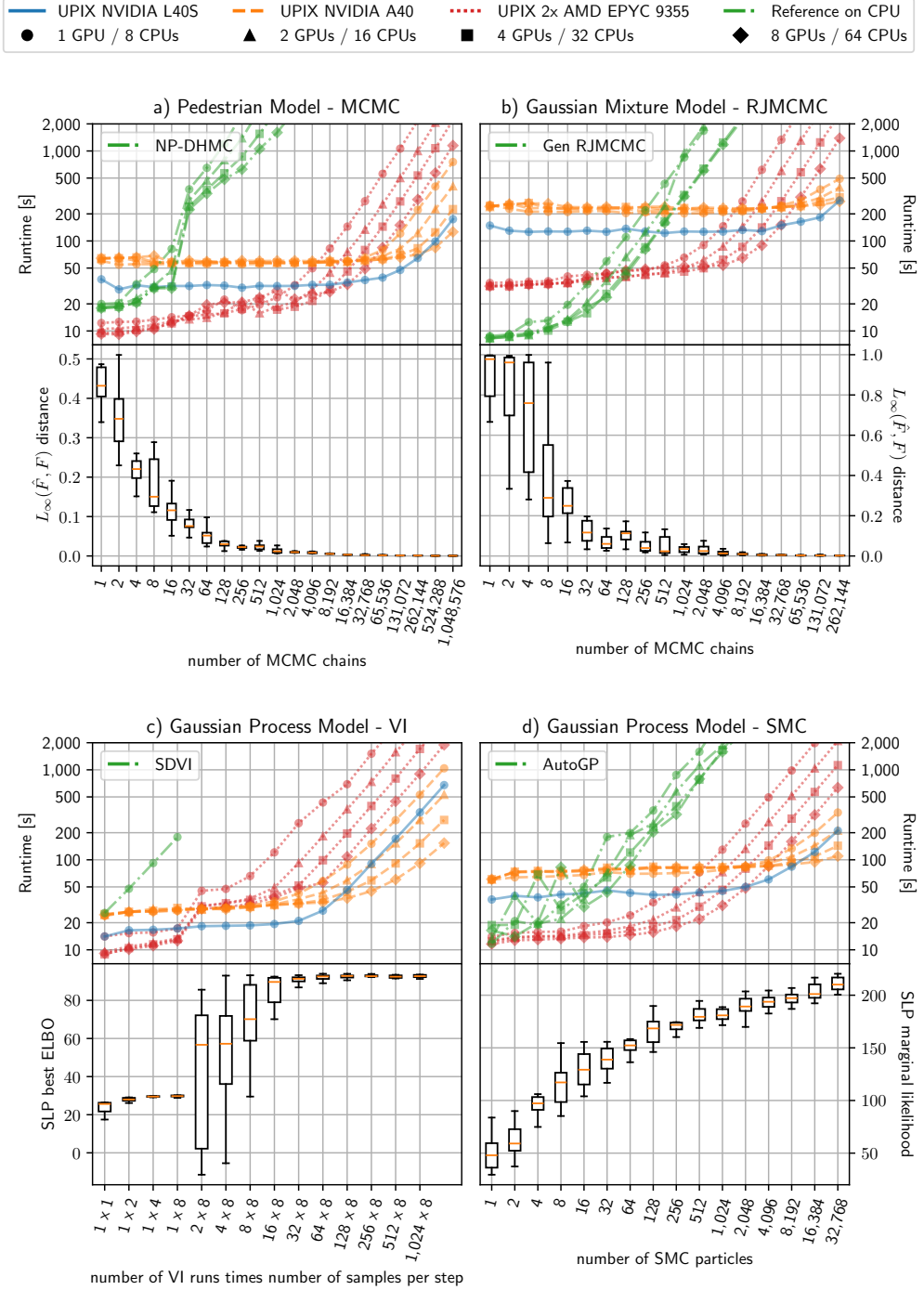


Fig. 8. Scaling experiments. Top plots: runtime of UPIX DCC algorithms and reference implementations on various hardware. Bottom plots: quality of posterior approximation. For each model, the same set of SLPs was used for all measurements to ensure a fair comparison.

First, we note that DCC algorithms can be straightforwardly parallelised by running inference for multiple SLPs in a multi-processing fashion on multiple CPU cores or multiple accelerator devices. This parallelisation method is expected to scale perfectly for models which require the exploration of many SLPs. Although this parallelisation method is supported in UPIX, conducting an experiment to evaluate it would offer limited value. Instead, we examine how inference algorithms executed on individual SLPs may benefit from the use of multiple accelerators. Note that since models with static support structure are also expressible in UPIX and yield a single SLP, the experimental results are likewise applicable to this smaller class of models. Figure 8 shows the complete experiment results which we discuss in detail in the following sections.

5.1 Scaling MCMC

In MCMC algorithms, each sample depends on the predecessor in the chain making them inherently sequential. Parallelisation can only be achieved when running multiple, typically 2-4, long chains at once. Recently, the many-short-chain approach has gained more attention [27, 53], where instead of approximating the posterior with long auto-correlated chains, it is approximated with thousands of short chains, sometimes using only their independent end-points. As MCMC algorithms are only asymptotically correct, it remains an open question how short the chains may be [36, 37].

We test this approach in UPIX by running MCMC-DCC for the Pedestrian model with up to $2^{20} = 1\,048\,576$ chains of 256 samples each and RJ-DCC for the GMM model with up to $2^{18} = 262\,144$ chains of 2028 samples each, using only their endpoints in the final approximation. We ran DCC ten times with different RNG seeds and make the runs comparable by using a fixed set of 8 SLPs. We show the L_∞ distance to the ground truth posterior in Figure 8 a) and b) on the bottom. As expected, the approximation quality improves with increasing number of chains.

On the top plots, we also report runtime measurements on various hardware. We observe close to perfect scaling: doubling the number of chains doubles the runtime, doubling the number of CPU cores / GPU devices halves the runtime. Due to the longer compile time, GPUs only outperform CPUs when the number of chains exceeds $2^{16} = 65\,536$. Only these large workloads benefit from using multiple GPUs. We note that the newer generation NVIDIA L40s GPUs compile JAX programs faster than the A40 GPUs, but unfortunately, we ran into device communication issues for multiple L40s devices and could not complete the experiments. We also report runtimes on CPU for the NP-DHMC and RJMCMC Gen implementation after which our DCC algorithms were modelled.

5.2 Scaling Variational Inference

Although Variational Inference has been scaled up to very large datasets [21], running VI on accelerators in an embarrassingly parallel way for smaller datasets has been less explored. At first glance, the parameter L , which controls the number samples taken to estimate the ELBO gradient in each step, seems like a good candidate for scaling: more accurate gradient estimates equals faster convergence. While this is true, fast convergence to a particular local maximum is often less desirable than having higher variance gradient estimates which help escape bad local maxima. We demonstrate this in Figure 9, where it can be seen that running VI eight times with $L = 8$ and picking the best run finds a better maxima compared to running VI once with $L = 64$.

Motivated by this finding, we examine this *multi-run VI* approach at large scale in Figure 8 c). In the lower plot, we show VI results for the Gaussian process SLP corresponding to kernel $(\text{PER} + \text{RQ}) \times \text{LIN}$ (best 3 node kernel), where we fix $L = 8$ and increase the number of parallel VI runs up to 2048. We performed 10 experiment repetitions and visualise the best ELBO achieved as boxplots. Note that in the VI run itself the ELBO is estimated with $L = 8$ samples, but for the plot we computed the ELBOs more accurately with 10 000 samples. It can be seen that with increasing number of runs better ELBOs are achieved more consistently.

As before, we measured the runtime of our multi-run SDVI approach with a fixed set of 8 SLPs on various hardware, see the upper plot in Figure 8 c). The scaling is again close to perfect. Notably, it is beneficial to use GPUs already for 2 parallel runs. For reference, we also measure the runtime of the PyTorch SDVI implementation [46] on CPU. This implementation does not support multiple parallel VI runs and does not implement the gradient estimate in vectorised fashion. Thus, we only show data for single runs and 8 CPUs, because we found that increasing the number of CPUs does not lower runtime. However, we reiterate that this SDVI implementation does support running inference for multiple SLPs in parallel in multiple processes.

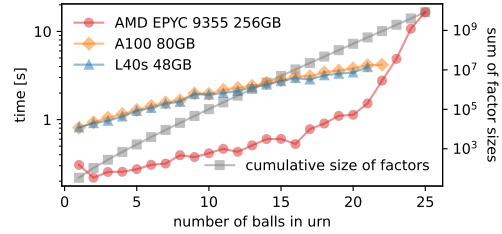
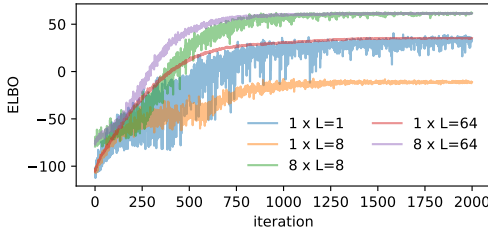


Fig. 9. ELBO curves for Variational Inference runs for Gaussian Process SLP (PER+RQ) $\times$ LIN with various L . Fig. 10. Cumulative size of factors versus inference runtime of variable elimination for the Urn model.

5.3 Scaling Sequential Monte Carlo

It is a well-known fact that increasing the number of particles in SMC improves approximation quality [11]. We confirmed this by running SMC 10 times for the Gaussian process SLP corresponding to kernel GE + (LIN $\times$ PER) (best 3 node kernel) for up to $2^{15} = 32\,768$ particles. As expected and shown in Figure 8 d), the marginal likelihood (normalisation constant) estimates increase and exhibit less variance with increasing number of particles.

The runtime of SMC-DCC measured for a fixed set of 8 SLPs scales close to perfectly again, as can be seen on the top plot in Figure 8 d). GPUs are preferable to CPUs with number of particles exceeding $2^{12} = 4\,096$. The runtime of AutoGP scales similarly to SMC-DCC in UPIX on CPUs. We have included these runtimes solely to have a reference for the scaling properties of UPIX. Although our SMC-DCC algorithm was inspired by AutoGP's methodology, the reported runtimes should not be interpreted as directly comparable, since the two approaches differ (see Section 4.4).

5.4 Scaling Variable Elimination

Variable elimination for discrete models is performed through repeated factor products. The product of factor f_1 with variables X_1 and factor f_2 with variables X_2 is implemented in log-space as sum of their tables given by arrays, where shared variables $x \in X_1 \cap X_2$ are placed at the same axis and axes corresponding to variables $x \notin X_1 \cap X_2$ are broadcasted. For models with high dimensional support this summation involves large arrays which can be efficiently computed on GPUs.

However, we found that variable elimination, tested on the Urn model, is memory bound. As can be seen in Figure 10, the size of the involved factors, and thus required memory, scales exponentially with N – the number of balls in the urn. This leads to out-of-memory errors at $N = 22$ for the 48GB A40 and L40s GPUs and at $N = 23$ for the 80GB A100 GPU. At these factor sizes, the computation time on GPU still exceeds the runtime of a CPU. In principle, we estimate that we could push N up to 24 or 25 by sharding the factors among multiple GPU devices or by unloading unused factors from the devices. As the accuracy of the posterior approximation is already close to numerical precision at $N = 23$, this would not significantly improve the inference quality and we leave such experiments for future work.

6 Limitations

6.1 Exploding Number of SLPs

By design, UPIX inherits the general limitations of the DCC approach. The DCC approach works well when the total number of sub-models / SLPs is low or when the observed data makes it possible to guide inference towards a small set of SLPs with high posterior mass. This was the case for all models considered: for the Pedestrian model we can reject SLPs corresponding to a number of steps greater than 6; for the GMM we can reject SLPs corresponding to a number of components greater than 11; for the Gaussian Process model the number of kernel structures is large, but most of them do not fit the data well; for the Urn model, we can enumerate all 20 relevant SLPs.

We briefly discuss a model for which the DCC approach fails. In statistical phylogenetic analysis [49], birth-death models provide a means to infer properties of the evolutionary tree of a given species. e.g., the birth or death rate of lineages. Data is inherently partial, lacking information about extinct lineages. Thus, it is the job of inference to consider many *unobserved lineages* and evolutionary trees that may explain the partial observed data. In DCC, this leads to an exploding number of SLPs, none of which can be easily rejected based on the partial observations.

6.2 Compilation Time

UPIX builds on the JIT-compilation machinery of JAX. Depending on model complexity, inference algorithm complexity, and hardware, this leads to substantial runtime overhead. In Table 2, we report our estimated compilation overhead for the evaluation in Section 4 on consumer hardware. In our scaling experiments of Section 5, we showed that for GPUs the computation time exceeds compile time only for large workloads for most models. Nevertheless, we demonstrated that our method, including compilation, achieves up to 960 times more computation within the same time budget, compared to existing approaches. Furthermore, compilation overhead is outweighed by the fact that UPIX enables scaling of inference algorithms on GPUs to workloads that are impractically slow on CPUs and existing approaches. Finally, future work on iterative compilation in JAX may further reduce compilation overhead.

Table 2. Estimated compilation overhead as fraction of runtime for all considered models.

Pedestrian	GP-VI	GMM	GP-SMC	Urn
21%	14%	39%	13%	48%

7 Related Work

7.1 Universal PPLs and Programmable Inference

With the release of Church, Goodman et al. [17] was the first to coin the term *universal PPL*. Since then, many universal PPLs were developed including WebPPL [18], Anglican [54], Pyro [7], and Turing [15]. The universal PPL Venture [35] and its successor Gen [13] are projects which pioneered programmable inference. UPIX adapts many established programmable inference ideas which have been developed for MCMC [35], variational inference [5], sequential Monte Carlo [30], and involutive MCMC [12]. It also supports the specification of variational distributions as guide programs, an approach popularised by Pyro [7]. While some components of the inference machinery in the aforementioned systems can be run on GPUs, none support compiling a full programmable inference routine to accelerator hardware for models with stochastic support, such as UPIX.

7.2 Inference for Models with Stochastic Support Structure

Many inference algorithms, like light-weight Metropolis Hastings algorithm [57], do not require assumptions about the support of the target model. This makes them general purpose, but also less efficient. When we can put assumptions on the model, like finite-dimensionality and differentiability, general-purpose algorithms are outperformed by specialised algorithms like NUTS [22]. There are efforts to extend these specialised algorithms to stochastic support models, e.g., non-parametric HMC [33] or non-parametric involutive MCMC [34].

The Divide-Conquer-Combine (DCC) approach [59] provides a technique to run inference algorithms, specialised for static support structures, on models specified in a universal PPL. This approach is under-explored with only one extension to variational inference called SDVI [47]. As the implementation of the original publication is closed source, there is only one open source implementation of the DCC approach by Reichelt et al. [46] in Pyro [7] (which we compared against in Sections 4 and 5) with a rudimentary translation to NumPyro [45]. UPIX is the first system to realise the DCC approach as a framework for formulating new DCC-based algorithms.

7.3 Accelerated Inference in Probabilistic Programming Systems

Compiling inference algorithms designed for universal PPLs to accelerator hardware like GPUs or TPUs is challenging. UPIX achieves this compilation by building on the DCC approach, but there also exist other techniques. To the best of our knowledge, other than UPIX, Lundén et al. [32] presents the only approach capable of compiling a full inference routine for a universal PPL to GPU. However, they solely focus on sequential Monte Carlo.

In general, GPU support is relatively limited in probabilistic programming. We highlight several systems that provide GPU support but, unlike universal PPLs, achieve this by adopting more restricted modelling languages. LibBi [41] is a language for Bayesian state-space modelling that features an OpenMP and CUDA backend for their sequential Monte Carlo implementation. Augur [24] is a probabilistic programming system that enables compilation of MCMC algorithms to CPU and GPU. An OpenCL backend was developed for Stan in 2019 [10]. Systems with a JAX backend like NumPyro [45], BlackJax [8], and TensorflowProbability [27], enable inference of finite-dimensional models with static support (i.e., no stochastic control flow or dynamic array shapes) on GPUs and TPUs. ProbZelus, a reactive probabilistic programming language, was also equipped with a JAX backend [2]. Most recently, GenJAX, a JAX backend for Gen, was developed and supports stochastic branching in a more restricted way by eagerly evaluating branches [4].

It is more common to use GPU acceleration in *deep probabilistic programming* where models incorporate neural networks. For instance, to compute the log-probability density function, systems like Edward [55] or Pyro [7] can dispatch the neural network computations to GPUs.

8 Conclusion

We presented UPIX, the first probabilistic programming system that realises the divide-conquer-combine (DCC) approach as a programmable inference framework for universal PPLs. This system enables not only the implementation of existing DCC algorithms, like the original DCC and SDVI, but also the formulation of new DCC algorithms, as was demonstrated by introducing RJ-DCC, SMC-DCC, and VE-DCC. By building on the JAX infrastructure, JIT-compilation to CPUs, GPUs, and TPUs was achieved. On a consumer-grade CPU, this resulted in up to 960 times more computation within the same time budget, compared to prior methods, which can greatly improve approximation quality for challenging probabilistic models. Lastly, in an empirical evaluation on machines up to 64 CPU cores and 8 GPU devices, we showed how inference algorithms can be scaled in UPIX to workloads that are impractically slow for CPUs and existing approaches.

Data-Availability Statement

The implementation of UPIX and the scripts to reproduce the evaluation of Section 4 and Section 5 are available at Anonymous [1]. This repository will also be submitted to artifact evaluation.

References

- [1] Anonymous. 2025. UPIX: Universal Programmable Inference in JAX. <https://anonymous.4open.science/r/UPIX-A583/>.
- [2] Guillaume Baudart, Louis Mandel, and Reyyan Tekin. 2022. Jax based parallel inference for reactive probabilistic programming. In *Proceedings of the 23rd ACM SIGPLAN/SIGBED International Conference on Languages, Compilers, and Tools for Embedded Systems*. 26–36.
- [3] Atilim Güneş Baydin, Lei Shao, Wahid Bhimji, Lukas Heinrich, Lawrence Meadows, Jialin Liu, Andreas Munk, Saeid Naderiparizi, Bradley Gram-Hansen, Gilles Louppe, et al. 2019. Etalumis: Bringing probabilistic programming to scientific simulators at scale. In *Proceedings of the international conference for high performance computing, networking, storage and analysis*. 1–24.
- [4] McCoy R Becker, Mathieu Huot, George Matheos, Xiaoyan Wang, Karen Chung, Colin Smith, Sam Ritchie, Rif A Saurous, Alexander K Lew, Martin C Rinard, et al. 2026. Probabilistic Programming with Vectorized Programmable Inference. *Proceedings of the ACM on Programming Languages* 10, POPL (2026), 2523–2554.
- [5] McCoy R Becker, Alexander K Lew, Xiaoyan Wang, Matin Ghavami, Mathieu Huot, Martin C Rinard, and Vikash K Mansinghka. 2024. Probabilistic programming with programmable variational inference. *Proceedings of the ACM on Programming Languages* 8, PLDI (2024), 2123–2147.
- [6] Michael Betancourt. 2017. A conceptual introduction to Hamiltonian Monte Carlo. *arXiv preprint arXiv:1701.02434* (2017).
- [7] Eli Bingham, Jonathan P Chen, Martin Jankowiak, Fritz Obermeyer, Neeraj Pradhan, Theofanis Karaletsos, Rohit Singh, Paul Szerlip, Paul Horsfall, and Noah D Goodman. 2019. Pyro: Deep universal probabilistic programming. *The Journal of Machine Learning Research* 20, 1 (2019), 973–978.
- [8] Alberto Cabezas, Adrien Corenflos, Junpeng Lao, and Rémi Louf. 2024. BlackJAX: Composable Bayesian inference in JAX. [arXiv:2402.10797](https://arxiv.org/abs/2402.10797) [cs.MS]
- [9] Bob Carpenter, Andrew Gelman, Matthew D Hoffman, Daniel Lee, Ben Goodrich, Michael Betancourt, Marcus Brubaker, Jiqiang Guo, Peter Li, and Allen Riddell. 2017. Stan: A probabilistic programming language. *Journal of statistical software* 76, 1 (2017).
- [10] Rok Češnovar, Steve Bronder, Davor Sluga, Jure Demšar, Tadej Ciglar, Sean Talts, and Erik Štrumbelj. 2019. GPU-based parallel computation support for Stan. *arXiv preprint arXiv:1907.01063* (2019).
- [11] Nicolas Chopin, Omiros Papaspiliopoulos, et al. 2020. *An introduction to sequential Monte Carlo*. Vol. 4. Springer.
- [12] Marco Cusumano-Towner, Alexander K Lew, and Vikash K Mansinghka. 2020. Automating involutive MCMC using probabilistic and differentiable programming. *arXiv preprint arXiv:2007.09871* (2020).
- [13] Marco F Cusumano-Towner, Feras A Saad, Alexander K Lew, and Vikash K Mansinghka. 2019. Gen: a general-purpose probabilistic programming system with programmable inference. In *Proceedings of the 40th acm sigplan conference on programming language design and implementation*. 221–236.
- [14] Roy Frostig, Matthew James Johnson, and Chris Leary. 2019. Compiling machine learning programs via high-level tracing. In *SysML conference 2018*.
- [15] Hong Ge, Kai Xu, and Zoubin Ghahramani. 2018. Turing: a language for flexible probabilistic inference. In *International conference on artificial intelligence and statistics*. PMLR, 1682–1690.
- [16] Diana Gibson and E Seneta. 1987. Augmented truncations of infinite stochastic matrices. *Journal of Applied Probability* 24, 3 (1987), 600–608.
- [17] Noah Goodman, Vikash Mansinghka, Daniel M Roy, Keith Bonawitz, and Joshua B Tenenbaum. 2012. Church: a language for generative models. *arXiv preprint arXiv:1206.3255* (2012).
- [18] Noah D Goodman and Andreas Stuhlmüller. 2014. The Design and Implementation of Probabilistic Programming Languages. <http://dippl.org>. Accessed: 2025-3-20.
- [19] Peter J Green. 1995. Reversible jump Markov chain Monte Carlo computation and Bayesian model determination. *Biometrika* 82, 4 (1995), 711–732.
- [20] Nils Lid Hjort, Chris Holmes, Peter Müller, and Stephen G Walker. 2010. *Bayesian nonparametrics*. Vol. 28. Cambridge University Press.
- [21] Matthew D Hoffman, David M Blei, Chong Wang, and John Paisley. 2013. Stochastic variational inference. *the Journal of machine Learning research* 14, 1 (2013), 1303–1347.
- [22] Matthew D Hoffman, Andrew Gelman, et al. 2014. The No-U-Turn sampler: adaptively setting path lengths in Hamiltonian Monte Carlo. *J. Mach. Learn. Res.* 15, 1 (2014), 1593–1623.

- [23] Steven Holtzen, Guy Van den Broeck, and Todd Millstein. 2020. Scaling exact inference for discrete probabilistic programs. *Proceedings of the ACM on Programming Languages* 4, OOPSLA (2020), 1–31.
- [24] Daniel Huang, Jean-Baptiste Tristan, and Greg Morrisett. 2017. Compiling Markov chain Monte Carlo algorithms for probabilistic modeling. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation*. 111–125.
- [25] Daphne Koller and Nir Friedman. 2009. *Probabilistic graphical models: principles and techniques*. MIT press.
- [26] Alp Kucukelbir, Dustin Tran, Rajesh Ranganath, Andrew Gelman, and David M Blei. 2017. Automatic differentiation variational inference. *Journal of machine learning research* 18, 14 (2017), 1–45.
- [27] Junpeng Lao, Christopher Suter, Ian Langmore, Cyril Chimisov, Ashish Saxena, Pavel Sountsov, Dave Moore, Rif A Saurous, Matthew D Hoffman, and Joshua V Dillon. 2020. ttfp. mcmc: Modern Markov chain Monte Carlo tools built for modern hardware. *arXiv preprint arXiv:2002.01184* (2020).
- [28] Wonyeol Lee, Hangeol Yu, Xavier Rival, and Hongseok Yang. 2019. Towards verified stochastic variational inference for probabilistic programs. *Proceedings of the ACM on Programming Languages* 4, POPL (2019), 1–33.
- [29] Alexander K Lew, Matin Ghavamizadeh, Martin C Rinard, and Vikash K Mansinghka. 2023. Probabilistic programming with stochastic probabilities. *Proceedings of the ACM on Programming Languages* 7, PLDI (2023), 1708–1732.
- [30] Alexander K Lew, George Matheos, Tan Zhi-Xuan, Matin Ghavamizadeh, Nishad Gothoskar, Stuart Russell, and Vikash K Mansinghka. 2023. Smcp3: Sequential monte carlo with probabilistic program proposals. In *International conference on artificial intelligence and statistics*. PMLR, 7061–7088.
- [31] Fernando Llorente, Luca Martino, David Delgado, and Javier Lopez-Santiago. 2023. Marginal likelihood computation for model selection and hypothesis testing: an extensive review. *SIAM review* 65, 1 (2023), 3–58.
- [32] Daniel Lundén, Joey Ohman, Jan Kudlicka, Viktor Senderov, Fredrik Ronquist, and David Broman. 2022. Compiling Universal Probabilistic Programming Languages with Efficient Parallel Sequential Monte Carlo Inference.. In *ESOP*. 29–56.
- [33] Carol Mak, Fabian Zaiser, and Luke Ong. 2021. Nonparametric hamiltonian monte carlo. In *International Conference on Machine Learning*. PMLR, 7336–7347.
- [34] Carol Mak, Fabian Zaiser, and Luke Ong. 2022. Nonparametric involutive markov chain monte carlo. In *International Conference on Machine Learning*. PMLR, 14802–14859.
- [35] Vikash K Mansinghka, Ulrich Schaechtle, Shivam Handa, Alexey Radul, Yutian Chen, and Martin Rinard. 2018. Probabilistic programming with programmable inference. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation*. 603–616.
- [36] Charles C Margossian and Andrew Gelman. 2023. For how many iterations should we run Markov chain Monte Carlo? *arXiv preprint arXiv:2311.02726* (2023).
- [37] Charles C Margossian, Matthew D Hoffman, Pavel Sountsov, Lionel Riou-Durand, Aki Vehtari, and Andrew Gelman. 2024. Nested ^R: Assessing the convergence of Markov chain Monte Carlo when running many short chains. *Bayesian Analysis* 1, 1 (2024), 1–28.
- [38] George Matheos, Alexander K Lew, Matin Ghavamizadeh, Stuart Russell, Marco Cusumano-Towner, and Vikash Mansinghka. 2020. Transforming worlds: Automated involutive MCMC for open-universe probabilistic models. In *Third Symposium on Advances in Approximate Bayesian Inference*.
- [39] Nicholas Metropolis, Arianna W Rosenbluth, Marshall N Rosenbluth, Augusta H Teller, and Edward Teller. 1953. Equation of state calculations by fast computing machines. *The journal of chemical physics* 21, 6 (1953), 1087–1092.
- [40] Brian Milch, Bhaskara Marthi, David Sontag, Stuart Russell, Daniel L Ong, and Andrey Kolobov. 2005. Approximate inference for infinite contingent Bayesian networks. In *International Workshop on Artificial Intelligence and Statistics*. PMLR, 238–245.
- [41] Lawrence M Murray. 2015. Bayesian state-space modelling on high-performance hardware using LibBi. *Journal of Statistical Software* 67 (2015), 1–36.
- [42] Kirill Neklyudov, Max Welling, Evgenii Egorov, and Dmitry Vetrov. 2020. Involutive MCMC: a unifying framework. In *International Conference on Machine Learning*. PMLR, 7273–7282.
- [43] Akihiko Nishimura, David B Dunson, and Jianfeng Lu. 2020. Discontinuous Hamiltonian Monte Carlo for discrete parameters and discontinuous likelihoods. *Biometrika* 107, 2 (2020), 365–380.
- [44] Agostino Nobile and Alastair T Fearnside. 2007. Bayesian finite mixtures with an unknown number of components: The allocation sampler. *Statistics and Computing* 17, 2 (2007), 147–162.
- [45] Du Phan, Neeraj Pradhan, and Martin Jankowiak. 2019. Composable Effects for Flexible and Accelerated Probabilistic Programming in NumPyro. *arXiv preprint arXiv:1912.11554* (2019).
- [46] Tim Reichelt, Luke Ong, and Thomas Rainforth. 2022. Rethinking variational inference for probabilistic programs with stochastic support. *Advances in Neural Information Processing Systems* 35 (2022), 15160–15175.
- [47] Tim Reichelt, Luke Ong, and Tom Rainforth. 2024. Beyond Bayesian Model Averaging over Paths in Probabilistic Programs with Stochastic Support. In *International Conference on Artificial Intelligence and Statistics*. PMLR, 829–837.

- [48] Sylvia Richardson and Peter J Green. 1997. On Bayesian analysis of mixtures with an unknown number of components (with discussion). *Journal of the Royal Statistical Society Series B: Statistical Methodology* 59, 4 (1997), 731–792.
- [49] Fredrik Ronquist, Jan Kudlicka, Viktor Senderov, Johannes Borgström, Nicolas Lartillot, Daniel Lundén, Lawrence Murray, Thomas B Schön, and David Broman. 2021. Universal probabilistic programming offers a powerful approach to statistical phylogenetics. *Communications biology* 4, 1 (2021), 244.
- [50] Feras Saad, Brian Patton, Matthew Douglas Hoffman, Rif A Saurous, and Vikash Mansinghka. 2023. Sequential Monte Carlo learning for time series structure discovery. In *International Conference on Machine Learning*. PMLR, 29473–29489.
- [51] Feras A Saad, Marco F Cusumano-Towner, Ulrich Schaehtle, Martin C Rinard, and Vikash K Mansinghka. 2019. Bayesian synthesis of probabilistic programs for automatic data modeling. *Proceedings of the ACM on Programming Languages* 3, POPL (2019), 1–32.
- [52] John Salvatier, Thomas V Wiecki, and Christopher Fonnesbeck. 2016. Probabilistic programming in Python using PyMC3. *PeerJ Computer Science* 2 (2016), e55.
- [53] Pavel Sountsov, Colin Carroll, and Matthew D Hoffman. 2024. Running markov chain monte carlo on modern hardware and software. *arXiv preprint arXiv:2411.04260* (2024).
- [54] David Tolpin, Jan-Willem van de Meent, and Frank Wood. 2015. Probabilistic programming in Anglican. In *Machine Learning and Knowledge Discovery in Databases: European Conference, ECML PKDD 2015, Porto, Portugal, September 7-11, 2015, Proceedings, Part III* 15. Springer, 308–311. <https://probprog.github.io/anglican/>
- [55] Dustin Tran, Matthew D Hoffman, Rif A Saurous, Eugene Brevdo, Kevin Murphy, and David M Blei. 2017. Deep probabilistic programming. *arXiv preprint arXiv:1701.03757* (2017).
- [56] Christopher KI Williams and Carl Edward Rasmussen. 2006. *Gaussian processes for machine learning*. Vol. 2. MIT press Cambridge, MA.
- [57] David Wingate, Andreas Stuhlmüller, and Noah Goodman. 2011. Lightweight implementations of probabilistic programming languages via transformational compilation. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*. JMLR Workshop and Conference Proceedings, 770–778.
- [58] Yi Wu, Lei Li, Stuart Russell, and Rastislav Bodik. 2016. Swift: Compiled inference for probabilistic programming languages. *arXiv preprint arXiv:1606.09242* (2016).
- [59] Yuan Zhou, Hongseok Yang, Yee Whye Teh, and Tom Rainforth. 2020. Divide, conquer, and combine: a new inference strategy for probabilistic programs with stochastic support. In *International Conference on Machine Learning*. PMLR, 11534–11545.

A Formalisation of Core Probabilistic Programming Language

A.1 Preliminaries

Before describing the density semantics of Section 3.2 in more detail, we review the most important measure theoretic concepts assuming some familiarity from the reader. A *measurable space* is a set X endowed with a σ -algebra Σ_X , which is a collection of subsets of X closed under complement, countable unions, and countable intersections. A function f between two measurable spaces $f: (X, \Sigma_X) \rightarrow (Y, \Sigma_Y)$ is called *measurable* if for all $B \in \Sigma_Y$ it holds that $f^{-1}(B) \in \Sigma_X$. The set of all measurable functions from A to B is denoted as $\text{Meas}(A, B)$. A *measure* μ is a function from $\Sigma_X \rightarrow [0, \infty]$ such that $\mu(\emptyset) = 0$ and $\mu(\biguplus_{i=1}^{\infty} A_i) = \sum_{i=1}^{\infty} \mu(A_i)$ for disjoint $A_i \in \Sigma_X$. If $\mu(X) = 1$ then we call μ a *probability measure* and if $\mu(X) < 1$ a *sub-probability measure*. A triple (X, Σ_X, μ) is called *measure or (sub-)probability space*. A measure is called σ -finite if there exists a countable partition of X into sets X_i such that $\mu(X_i) < \infty$.

If F is a collection of subsets of X , we may generate a σ -algebra via intersection of all σ -algebras that contain F , i.e., $\sigma(F) = \bigcap_{F \subseteq \Sigma_X} \Sigma_X$ is the smallest σ -algebra on X that contains F . We may form finite products of measure spaces $(X_i, \Sigma_{X_i}, \mu_i)$ with σ -finite μ_i by equipping the cartesian product $\prod_{i=1}^n X_i$ with the σ -algebra generated by all rectangle set $\bigotimes_{i=1}^n \Sigma_{X_i} = \sigma(\prod_{i=1}^n A_i : A_i \in \Sigma_{X_i})$. The product measure is uniquely defined by $\bigotimes_{i=1}^n \mu_i(A_1 \times \dots \times A_n) = \mu_1(A_1) \cdot \dots \cdot \mu_n(A_n)$ for $A_i \in \Sigma_{X_i}$. We may also build sums of *disjoint* measure spaces by equipping $\biguplus_{i \in I} X_i$ with σ -algebra $\bigoplus_{i \in I} \Sigma_{X_i} = \{A \subseteq \biguplus_{i \in I} X_i : \forall i \in I : A \cap X_i \in \Sigma_{X_i}\}$, where I is a countably-infinite index set. The sum of measures is uniquely defined by $\bigoplus_{i \in I} \mu_i(\biguplus_{i \in I} A_i) = \sum_{i \in I} \mu_i(A_i)$ for $A_i \in \Sigma_{X_i}$. Lastly, we may also restrict measure spaces to a subset $Y \in \Sigma_X$ with $\Sigma_Y = \{A \cap Y : A \in \Sigma_X\}$ and $\mu_Y = \mu_X$.

A.2 Type System

For our type system, we equip each base type $\mathbb{G} = \{\mathbb{B}, \mathbb{N}, \mathbb{Z}, \mathbb{R}\}$ with a measure space. The discrete types $\mathbb{B}$, $\mathbb{N}$, and $\mathbb{Z}$ are equipped with the *discrete* σ -algebra which is just the collection of all subsets and the *counting measure* $\xi(A) = |A|$. Note that in our type system we assume disjoint sets, e.g., $\mathbb{N} \cap \mathbb{Z} = \emptyset$. The real numbers are equipped with the *Borel* σ -algebra $\mathcal{B} = \sigma(\{(-\infty, r) : r \in \mathbb{R}\})$ and the *Lebesgue measure* λ which is uniquely defined by $\lambda((a, b)) = b - a$. Array types $G[s_1 \dots s_n] \in \mathbb{A}$ are given by the product space $\prod_{i=1}^{s_1 \dots s_n} (G, \Sigma_G, \mu_G)$. The set of all values $\text{Val} = \biguplus \mathbb{T}$, where $\mathbb{T} = \mathbb{G} \uplus \mathbb{A}$, is the sum measure spaces of all (disjoint) type measure spaces $\biguplus_{T \in \mathbb{T}} (T, \Sigma_T, \mu_T)$. For a command with n variables, the set of states $\mathcal{S} = [\text{Var} \rightarrow \text{Val}] \cong \prod_{i=1}^n \text{Val}$ is equipped with the product space $(\prod_{i=1}^n \text{Val}, \bigotimes_{i=1}^n \Sigma_{\text{Val}}, \bigotimes_{i=1}^n \mu_{\text{Val}})$. The variables Var include the reserved variables $\mathbf{p}$ and $\mathbf{w}$ for accumulating the density and the variable $\mathbf{b}$ for tracking "concretization mismatches".

To define the space of traces, we first define the finite set of *trace shapes of length n* as ordered lists of pairs $\text{Str} \times \mathbb{A}$, $\mathbb{S}_n = \{(s_1, T_1), \dots, (s_n, T_n) : T_i \in \mathbb{A}, s_i \in \text{Str}, s_i < s_{i+1}\}$, and the countably infinite set of all trace shapes as $\mathbb{S} = \biguplus_{i=0}^{\infty} \mathbb{S}_n$. Let $\text{shape}(\text{tr})$ be the shape of trace tr . For each trace shape $s \in \mathbb{S}$ we can build the product measure space $M_s = (\prod_{(\alpha, A) \in s} A, \bigotimes_{(\alpha, A) \in s} \Sigma_A, \bigotimes_{(\alpha, A) \in s} \mu_A)$ to obtain a measure space on traces by their union $(\mathcal{T}, \Sigma_{\mathcal{T}}, \mu_{\mathcal{T}}) = \biguplus_{s \in \mathbb{S}} M_s$. This construction formalises the notion of stochastic support structure as the shape of traces may vary. In contrast, the measure space for a static support would be given by a single M_s for a fixed trace shape s . Note that the original formulation of the density semantics by Lee et al. [28] only considers $\biguplus_{K \in \text{C}_{\text{fin}} \text{Str}} [K \rightarrow \mathbb{R}]$ as the space for traces and $[\text{Var} \rightarrow \mathbb{R}]$ as space for states.

A.3 Primitive Functions, Distributions, and Type Checking

We also extended the semantics of Lee et al. [28] with dynamic type checks. Primitive functions f and g in the language are interpreted as measurable functions. Primitive numeric-valued functions are measurable functions that map from a product space to a sum-space, i.e., $\llbracket f \rrbracket : T_1 \times \dots \times T_n \rightarrow A$,

where $T_i \in \mathbb{T}$ and $A = \biguplus_{T \in U} T$ for some $U \subseteq \mathbb{T}$. The measure spaces are constructed as explained in the previous sections. We also require that if f accepts an array input that it *must* produce an array output. Thus, we ensure that there is no function that maps arrays to scalars. In summary,

$$\llbracket f \rrbracket \in \bigcup \{ \text{Meas}(T_1 \times \dots \times T_n, A) : n \geq 0 \wedge T_i \in \mathbb{T} \wedge ((\exists i : T_i \in \mathbb{A}) \Rightarrow A \subseteq \biguplus \mathbb{A}) \}.$$

The primitive string-valued functions g operate only on scalar inputs, i.e.,

$$\llbracket g \rrbracket \in \bigcup \{ \text{Meas}(\mathbf{Str}^m \times T_1 \times \dots \times T_n, \mathbf{Str}) : m \geq 0 \wedge n \geq 0 \wedge T_i \in \mathbb{G} \}.$$

The set of strings is equipped with the discrete σ -algebra and counting measure.

Distributions primitives $d(E_1, \dots, E_n)$ are defined by their density which are measurable functions $\text{pdf}_d \in \text{Meas}(T_0 \times T_1 \times \dots \times T_n, [0, \infty))$, where $[0, \infty)$ is equipped with the Borel sets restricted to the domain via intersection and the Lebesgue measure. We define $\text{supp}(d) = T_0$.

Now we are ready to define the dynamic type errors with $\text{recTE}(\sigma, E, T)$ recursively:

$$\begin{aligned} \text{recTE}(\sigma, c, T) &= c \notin T & \text{recTE}(\sigma, \alpha, \mathbf{Str}) &= \text{true} & \text{recTE}(\sigma, x, T) &= \sigma(x) \notin T \\ \text{recTE}(\sigma, f(E_1, \dots, E_n), T) &= \bigvee_{i=1}^n \text{recTE}(\sigma, E_i, T_i) \vee A \not\subseteq T \text{ for } \llbracket f \rrbracket \in \text{Meas}(T_1 \times \dots \times T_n, A) \\ \text{recTE}(\sigma, g(S_1, \dots, S_m, E_1, \dots, E_n), \mathbf{Str}) &= \bigvee_{i=1}^m \text{recTE}(\sigma, S_i, \mathbf{Str}) \vee \bigvee_{i=1}^n \text{recTE}(\sigma, E_i, T_i) \\ &\quad \text{for } \llbracket g \rrbracket \in \text{Meas}(\mathbf{Str}^m \times T_1 \times \dots \times T_n, \mathbf{Str}) \\ \text{typeerr}(\sigma, E) &= \text{recTE}(\sigma, E, \text{Val}) \\ \text{typeerr}(\sigma, S, d(E_1, \dots, E_n)) &= \text{recTE}(\sigma, S, \mathbf{Str}) \vee \bigvee_{i=1}^n \text{recTE}(\sigma, E_i, T_i) \\ \text{typeerr}(\sigma, S, d(E_1, \dots, E_n), E_{n+1}) &= \text{typeerr}(\sigma, S, d(E_1, \dots, E_n)) \vee \text{recTE}(\sigma, E_{n+1}, T_0) \\ &\quad \text{for } \text{pdf}_d \in \text{Meas}(T_0 \times T_1 \times \dots \times T_n, [0, \infty)) \end{aligned}$$

A.4 Denotational Density Semantics

In this section, we repeat the definition of the semantics and prove well-definedness. Recall, commands C are interpreted as measurable functions $\llbracket C \rrbracket : \{\perp\} \uplus (\mathcal{S} \times \mathcal{T}) \rightarrow \{\perp\} \uplus (\mathcal{S} \times \mathcal{T})$:

$$\begin{aligned} \llbracket \text{skip} \rrbracket(\sigma, \text{tr}) &= (\sigma, \text{tr}) \\ \llbracket x = E \rrbracket(\sigma, \text{tr}) &= \text{if } (\text{typeerr}(\sigma, E)) \text{ then } \perp \text{ else } (\sigma[x \mapsto \llbracket E \rrbracket \sigma], \text{tr}) \\ \llbracket C_1; C_2 \rrbracket(\sigma, \text{tr}) &= (\llbracket C_2 \rrbracket \circ \llbracket C_1 \rrbracket)(\sigma, \text{tr}) \\ \llbracket \text{if } E \text{ then } C_1 \text{ else } C_2 \rrbracket(\sigma, \text{tr}) &= \text{if } (\text{typeerr}(\sigma, E) \vee \llbracket E \rrbracket \sigma \notin \mathbb{B}) \text{ then } \perp \\ &\quad \text{else if } (\llbracket E \rrbracket \sigma = \text{true}) \text{ then } \llbracket C_1 \rrbracket(\sigma, \text{tr}) \text{ else } \llbracket C_2 \rrbracket(\sigma, \text{tr}) \\ \llbracket \text{while } E \text{ do } C \rrbracket(\sigma, \text{tr}) &= (\text{fix } G)(\sigma, \text{tr}), \\ &\quad \text{where } G(g)(\sigma, \text{tr}) = \text{if } (\text{typeerr}(\sigma, E) \vee \llbracket E \rrbracket \sigma \notin \mathbb{B}) \text{ then } \perp \\ &\quad \text{else if } (\llbracket E \rrbracket \sigma = \text{true}) \text{ then } (g \circ \llbracket C \rrbracket)(\sigma, \text{tr}) \text{ else } \llbracket \text{skip} \rrbracket(\sigma, \text{tr}) \\ \llbracket x = \text{sample}(S, d(E_1, \dots, E_n)) \rrbracket(\sigma, \text{tr}) &= \\ &\quad \text{if } (\text{typeerr}(\sigma, S, d(E_1, \dots, E_n)) \vee \llbracket S \rrbracket \sigma \notin \text{dom}(\text{tr}) \vee \text{tr}(\llbracket S \rrbracket \sigma) \notin \text{supp}(d)) \text{ then } \perp \\ &\quad \text{else } (\sigma[x \mapsto \text{tr}(\llbracket S \rrbracket \sigma)], \mathbf{p} \mapsto \sigma(\mathbf{p}) \cdot \text{pdf}_d(\text{tr}(\llbracket S \rrbracket \sigma), \llbracket E_1 \rrbracket \sigma, \dots, \llbracket E_n \rrbracket \sigma), \text{tr} \setminus \llbracket S \rrbracket \sigma) \\ \llbracket \text{score}(S, d(E_1, \dots, E_n), E_{n+1}) \rrbracket(\sigma, \text{tr}) &= \\ &\quad \text{if } (\text{typeerr}(\sigma, S, d(E_1, \dots, E_n), E_{n+1}) \vee \llbracket S \rrbracket \sigma \in \text{dom}(\text{tr}) \vee \llbracket E_{n+1} \rrbracket \sigma \notin \text{supp}(d)) \text{ then } \perp \\ &\quad \text{else } (\sigma[\mathbf{w} \mapsto \sigma(\mathbf{w}) \cdot \text{pdf}_d(\llbracket E_{n+1} \rrbracket \sigma, \llbracket E_1 \rrbracket \sigma, \dots, \llbracket E_n \rrbracket \sigma)], \text{tr}) \\ \llbracket x = \text{concretize}(E) \rrbracket(\sigma, \text{tr}) &= \text{if } (\text{typeerr}(\sigma, E)) \text{ then } \perp \\ &\quad \text{else if } (\exists G \in \mathbb{G} : \llbracket E \rrbracket \sigma \in G[]) \text{ then } (\sigma[x \mapsto \iota(\llbracket E \rrbracket \sigma)], \text{tr}) \text{ else } \perp \end{aligned}$$

We use the standard notation for state updates $\sigma[x \mapsto v]$. In contrast to Lee et al. [28], we keep track of the density computation via the reserved state variables $\mathbf{p}$ and $\mathbf{w}$ to ease presentation. We also extend the domain of $\llbracket \cdot \rrbracket$ with $\perp$, such that $\llbracket C \rrbracket(\perp) = \perp$ for all commands C , to make the interpretation composable without a lifting function. Recall, we use $\beta \in \text{dom}(\text{tr})$ to indicate that $\exists i, B: s_i = (\beta, B)$, where s is the shape of the trace: $\text{tr} \in \prod_{(\alpha, A) \in s} A$ and write $\text{tr}(\beta) = \text{tr}_i$ in this case. We denote the "deletion" of address β from trace tr as $\text{tr} \setminus \beta$.

Lastly, the interpretation of expressions is standard:

$$\begin{aligned} \llbracket c \rrbracket \sigma &= c \\ \llbracket x \rrbracket \sigma &= \sigma(x) \\ \llbracket f(E_1, \dots, E_n) \rrbracket \sigma &= \llbracket f \rrbracket(\llbracket E_1 \rrbracket \sigma, \dots, \llbracket E_n \rrbracket \sigma) \\ \llbracket \alpha \rrbracket \sigma &= \alpha \\ \llbracket g(S_1, \dots, S_m, E_1, \dots, E_n) \rrbracket \sigma &= \llbracket g \rrbracket(\llbracket S_1 \rrbracket \sigma, \dots, \llbracket S_m \rrbracket \sigma, \llbracket E_1 \rrbracket \sigma, \dots, \llbracket E_n \rrbracket \sigma) \end{aligned}$$

We have following property:

LEMMA 1. *For all expressions E and S , their semantics $\llbracket E \rrbracket: \mathcal{S} \rightarrow \text{Val}$ and $\llbracket S \rrbracket: \mathcal{S} \rightarrow \text{Str}$ are measurable functions.*

PROOF. Follows exactly as [28, Lemma 4.1]. $\square$

LEMMA 2. *For all expressions E and types T , the type-checking function $\sigma \mapsto \text{recTE}(\sigma, E, T)$ is measurable from $\mathcal{S}$ to $\mathbb{B}$. Thus, also $\sigma \mapsto \text{typeerr}(\sigma, E)$, $\sigma \mapsto \text{typeerr}(\sigma, S, d(E_1, \dots, E_n))$, and $\sigma \mapsto \text{typeerr}(\sigma, S, d(E_1, \dots, E_n), E_{n+1})$ are measurable for all distributions d and expressions S, E_i .*

PROOF. Follows via structural induction from Lemma 1 and the fact that set membership tests are measurable in sum measure spaces. $\square$

We define following set

$$\mathcal{D} = \{g \in \{\perp\} \uplus (\mathcal{S} \times \mathcal{T}) \rightarrow \{\perp\} \uplus (\mathcal{S} \times \mathcal{T}) : g \text{ is measurable, strict and local}\},$$

where $\{\perp\}$ is equipped with the discrete σ -algebra for measurability, strict means $g(\perp) = \perp$ and locality ensures that the traces are consumed properly (only relevant for the connection in Appendix C). The function g is local if and only if

$$\begin{aligned} g(\sigma, \text{tr}) = (\sigma', \text{tr}') &\implies (\exists \Delta \in \mathcal{T} : \Delta \# \text{tr}' \wedge \text{tr} = \text{tr}' \uplus \Delta \wedge g(\sigma, \Delta) = (\sigma', \{\})) \\ &\quad \wedge (\forall \Delta \in \mathcal{T} : \Delta \# \text{tr}' \implies g(\sigma, \text{tr} \uplus \Delta) = (\sigma', \text{tr}' \uplus \Delta)), \end{aligned}$$

where $\Delta \# \text{tr}$ indicates disjoint traces $\Delta \# \text{tr} \iff (\text{dom}(\Delta) \cap \text{dom}(\text{tr}) = \emptyset)$ with the trace domain defined as $\text{dom}(\text{tr}) = \text{dom}(s) = \{\alpha \in \text{Str} : \exists A \in \mathbb{A} : (\alpha, A) \in s\}$ for trace shape $s = \text{shape}(\text{tr})$.

We define a partial order on $\mathcal{D}$ with

$$g \sqsubseteq g' = \forall a \in \mathcal{S} \times \mathcal{T} : g(a) = \perp \vee g(a) = g'(a).$$

LEMMA 3. *$(\mathcal{D}, \sqsubseteq)$ is an ω -complete partial order and has the least element $a \mapsto \perp$.*

PROOF. Follows essentially by the same proof as [28, Lemma 4.5]. $\square$

LEMMA 4. *The semantic function $\llbracket \cdot \rrbracket$ is well-defined and $\llbracket C \rrbracket \in \mathcal{D}$ for all commands C .*

PROOF. We adapt the structural induction proof of [28, Lemma 4.6] with minor adaptations.

We omit the proofs for skip, assignment, sequence, and if, as these follow the original arguments while incorporating the measurability of type checking. The same is true for the while statement.

The fixed-point construction is well-defined since G can be shown to be continuous as in the original proof. The fact that $\mathcal{D}$ is a ω -cpo implies that G has a unique least fixed point.

We continue with the sample command

$$\begin{aligned} \llbracket x = \text{sample}(S, d(E_1, \dots, E_n)) \rrbracket(\sigma, \text{tr}) = \\ \text{if } (\text{typeerr}(\sigma, S, d(E_1, \dots, E_n)) \vee \llbracket S \rrbracket \sigma \notin \text{dom}(\text{tr}) \vee \text{tr}(\llbracket S \rrbracket \sigma) \notin \text{supp}(d)) \text{ then } \perp \\ \text{else } (\sigma[x \mapsto \text{tr}(\llbracket S \rrbracket \sigma)], \mathbf{p} \mapsto \sigma(\mathbf{p}) \cdot \text{pdf}_d(\text{tr}(\llbracket S \rrbracket \sigma), \llbracket E_1 \rrbracket \sigma, \dots, \llbracket E_n \rrbracket \sigma), \text{tr} \setminus \llbracket S \rrbracket \sigma). \end{aligned}$$

For $\beta \in \text{dom}(\text{tr})$, the restriction operation for trace $\text{tr} = \prod_{(\alpha, A) \in s} v_\alpha$ with shape s is defined as $\text{tr} \setminus \beta = \prod_{(\alpha, A) \in s, \alpha \neq \beta} v_\alpha$ giving rise to a trace with a new shape. In the case of $\llbracket S \rrbracket \sigma \in \text{dom}(\text{tr})$, it is clear that the sample command only consumes the trace at address $\llbracket S \rrbracket \sigma$ which implies locality with $\Delta = \{\llbracket S \rrbracket \sigma \mapsto \text{tr}(\llbracket S \rrbracket \sigma)\} \in \mathcal{T}$ and the fact that the trace is not modified for any addresses $\beta \neq \llbracket S \rrbracket \sigma$.

We use Lemma 1 to prove measurability. First, to show that by countable union of intersections of measurable sets, the following set is also measurable

$$D_S = \{(\sigma, \text{tr}) : \llbracket S \rrbracket \sigma \in \text{dom}(\text{tr})\} = \bigcup_{s \in \mathbb{S}} \{(\sigma, \text{tr}) : \text{shape}(\text{tr}) = s\} \cap \{(\sigma, \text{tr}) : \llbracket S \rrbracket \sigma \in \text{dom}(s)\}.$$

Next, it is straightforward to see that the application function $(\sigma, \text{tr}) \mapsto \text{tr}(\llbracket S \rrbracket \sigma)$ is measurable from $(\mathcal{S} \times \mathcal{T}) \cap D_S$ to Val and that the restriction function $(\sigma, \text{tr}) \mapsto \text{tr} \setminus \llbracket S \rrbracket \sigma$, being a kind of projection, is measurable from $(\mathcal{S} \times \mathcal{T}) \cap D_S$ to $\mathcal{T}$.

With these facts and by measurability of type-checking (Lemma 2),

$$V = \{(\sigma, \text{tr}) \in D_S : \llbracket S \rrbracket \sigma \in \text{dom}(\text{tr}) \wedge \neg \text{typeerr}(\sigma, S, d(E_1, \dots, E_n)) \wedge \text{tr}(\llbracket S \rrbracket \sigma) \in \text{supp}(d)\}$$

is a measurable set and $\llbracket x = \text{sample}(S, d(E_1, \dots, E_n)) \rrbracket^{-1}(\{\perp\}) = \{\perp\} \uplus (\mathcal{S} \times \mathcal{T}) \setminus V$ is also measurable. For all measurable subsets B of $\mathcal{S} \times \mathcal{T}$, we have

$$\begin{aligned} \llbracket \text{sample}(S, d(E_1, \dots, E_n)) \rrbracket^{-1}(B) = \\ \{(\sigma, \text{tr}) \in V : (\sigma[x \mapsto \text{tr}(\llbracket S \rrbracket \sigma)], \mathbf{p} \mapsto \sigma(\mathbf{p}) \cdot \text{pdf}_d(\text{tr}(\llbracket S \rrbracket \sigma), \llbracket E_1 \rrbracket \sigma, \dots, \llbracket E_n \rrbracket \sigma), \text{tr} \setminus \llbracket S \rrbracket \sigma) \in B\} \end{aligned}$$

It follows that this pre-image is measurable, since state update, multiplication, density pdf_d , trace application, and restriction are all measurable operations. We have proven measurability of the sample command on $\{\perp\}$ and $\mathcal{S} \times \mathcal{T}$ which implies measurability on $\{\perp\} \uplus (\mathcal{S} \times \mathcal{T})$.

The claim for the score command follows similarly to the proof for the sample command.

Lastly, the claim for concretization command,

$$\begin{aligned} \llbracket x = \text{concretize}(E) \rrbracket(\sigma, \text{tr}) = \text{if } (\text{typeerr}(\sigma, E)) \text{ then } \perp \\ \text{else if } (\exists G \in \mathbb{G} : \llbracket E \rrbracket \sigma \in G[]) \text{ then } (\sigma[x \mapsto \iota(\llbracket E \rrbracket \sigma)], \text{tr}) \text{ else } \perp, \end{aligned}$$

follows similarly from the measurability of type-checking, state updating, and the isomorphism ι . $\square$

Lastly, recall that the density function for a probabilistic model defined by the command C is the measurable function $p_C : \mathcal{T} \rightarrow [0, \infty)$ given by

$$p_C(\text{tr}) = \begin{cases} \sigma(\mathbf{p}) \cdot \sigma(\mathbf{w}) & \text{if } \exists \sigma \in \mathcal{S} : \llbracket C \rrbracket(\sigma_0, \text{tr}) = (\sigma, \{\}), \\ 0 & \text{otherwise,} \end{cases}$$

where $\sigma_0 \in \mathcal{S}$ is some fixed initial state with $\sigma(\mathbf{p}) = 1$ and $\sigma(\mathbf{w}) = 1$.

B Proof of Theorem 1

Before we continue with the proof of the main soundness theorem, we review the formalisation of the dynamic program analysis and the program transformation in more detail.

B.1 Tracked Types

Recall that for each array type $A \in \mathbb{A}$, we have a dual tracked version $A^\dagger \cong A$. These sets are isomorphic with $\text{val}: A^\dagger \rightarrow A$ and $\cdot^\dagger: A \rightarrow A^\dagger$, but otherwise taken to be disjoint. We extended the types $\mathbb{T}^* = \mathbb{G} \uplus \mathbb{A} \uplus \mathbb{A}^\dagger$, $\text{Val}^* = \uplus \mathbb{T}^*$, and states $\mathbb{S}^* = [\text{Var} \rightarrow \text{Val}^*]$. Tracked traces map strings to tracked arrays $\mathcal{T}^\dagger = \uplus_{s \in \mathbb{S}} \prod_{(\alpha, A) \in s} A^\dagger$. These types were introduced to track which values depend on the input trace. To this end we introduced new semantics for the function calls

$$\llbracket f(E_1 \dots, E_n) \rrbracket^\dagger \sigma = \begin{cases} \llbracket f \rrbracket(\text{val}^*(v_1), \dots, \text{val}^*(v_n))^\dagger & \text{if } (v_1, \dots, v_n) \in (\text{Val}^*)^n \setminus \text{Val}^n, \\ \llbracket f \rrbracket(v_1, \dots, v_n) & \text{otherwise,} \end{cases}$$

where $v_i = \llbracket E_i \rrbracket^\dagger \sigma \in \text{Val}^*$ and we defined $\text{val}^*(v) = (\text{if } (v \in \text{Val}) \text{ then } v \text{ else } \text{val}(v))$. The distribution densities are updated to $\text{pdf}_d^\dagger$ similarly, but always return untracked values, and $\llbracket S \rrbracket^\dagger = \llbracket S \rrbracket$ for string expressions. With this definition it holds that the output of a function call is tracked if any of its arguments is tracked, i.e., for a well-typed function application we have

$$\exists i: \llbracket E_i \rrbracket^\dagger \sigma \in \uplus \mathbb{A}^\dagger \implies \llbracket f(E_1, \dots, E_n) \rrbracket^\dagger \sigma \in \uplus \mathbb{A}^\dagger.$$

More generally, we have following properties corresponding to the negation of above implication.

LEMMA 5. *For all expressions E and string expressions S , we have*

$$\neg \text{typeerr}(\sigma, E) \wedge \llbracket E \rrbracket^\dagger \sigma \in \text{Val} \implies \forall x \in \text{vars}(E): \sigma(x) \in \text{Val},$$

$$\neg \text{typeerr}(\sigma, S) \implies \forall x \in \text{vars}(S): \sigma(x) \in \text{Val},$$

where $\text{vars}(E)$ denotes all variables in E or S .

Next, we also have to update type checking to allow tracked array arguments to functions. We only state the relevant change below, all other rules remain the same.

$$\text{recTE}^\dagger(\sigma, x, T) = \begin{cases} \sigma(x) \notin T \uplus T^\dagger & \text{if } T \subseteq \uplus \mathbb{A}, \\ \sigma(x) \notin T & \text{otherwise.} \end{cases}$$

For the same reason, we extend the distribution support to $\text{supp}^\dagger(d) = \text{supp}(d) \uplus \text{supp}(d)^\dagger$.

B.2 Semantics of Dynamic Program Analysis

In the semantics of the program analysis $\llbracket \cdot \rrbracket^\dagger: \{\perp\} \uplus (\mathbb{S}^* \times \mathcal{T}^\dagger \times \mathcal{L}) \rightarrow \{\perp\} \uplus (\mathbb{S}^* \times \mathcal{T}^\dagger \times \mathcal{L})$ we introduced the list argument $\ell \in \mathcal{L}$. The measure space of lists with scalar values is constructed similarly to traces $\mathcal{L} = \uplus_{n \geq 0, G_i \in \mathbb{G}} \prod_{i=1}^n G_i$. From Lemma 3 it is straightforward to see that the corresponding function space

$$\mathcal{D}^\dagger = \{g \in \{\perp\} \uplus (\mathbb{S}^* \times \mathcal{T}^\dagger \times \mathcal{L}) \rightarrow \{\perp\} \uplus (\mathbb{S}^* \times \mathcal{T}^\dagger \times \mathcal{L}): g \text{ is measurable, strict and local}\}$$

still forms a ω -complete partial order.

The semantics are primarily new for the concretization command:

$$\begin{aligned} \llbracket x = \text{concretize}(E) \rrbracket^\dagger(\sigma, \text{tr}, \ell) = & \text{if } (\text{typeerr}^\dagger(\sigma, E)) \text{ then } \perp \\ & \text{else if } (\exists G \in \mathbb{G}: \llbracket E \rrbracket^\dagger \sigma \in G[])^\dagger \text{ then } (\sigma[x \mapsto \iota^\dagger(\llbracket E \rrbracket^\dagger \sigma)], \text{tr}, \ell.\text{append}(\iota^\dagger(\llbracket E \rrbracket^\dagger \sigma))) \\ & \text{else if } (\exists G \in \mathbb{G}: \llbracket E \rrbracket^\dagger \sigma \in G[]) \text{ then } (\sigma[x \mapsto \iota(\llbracket E \rrbracket^\dagger \sigma)], \text{tr}, \ell) \text{ else } \perp. \end{aligned}$$

All other commands leave the list ℓ unmodified. In the definitions of Appendix A.4, we simply need to replace $\llbracket \cdot \rrbracket$ with $\llbracket \cdot \rrbracket^\dagger$, $\text{typeerr}(\sigma, E)$ with $\text{typeerr}^\dagger(\sigma, E)$, $\text{supp}(d)$ with $\text{supp}^\dagger(d)$, and pdf_d with $\text{pdf}_d^\dagger$. Note that in the semantics of the if command and the while command, we still check that the condition evaluates to a boolean $\mathbb{B}$. This ensures that branching can only depend on the tracked trace via the concretize command. Thus, we record all branching decisions at these commands.

It follows immediately that tracked types preserve semantics, where $\text{tr}^\dagger \in \mathcal{T}^\dagger$ denotes the tracked version of $\text{tr} \in \mathcal{T}$:

LEMMA 6. For command C , trace $\text{tr} \in \mathcal{T}$, and state $\sigma \in \Sigma$, we have

$$\llbracket C \rrbracket(\sigma, \text{tr}) = \perp \iff \llbracket C \rrbracket^\dagger(\sigma, \text{tr}^\dagger, []) = \perp \text{ and }$$

$$\exists \sigma_1 \in \mathcal{S}: \llbracket C \rrbracket(\sigma, \text{tr}) = (\sigma_1, \{\}) \iff \exists \sigma_2 \in \mathcal{S}^*, \ell \in \mathcal{L}: \llbracket C \rrbracket^\dagger(\sigma, \text{tr}^\dagger, []) = (\sigma_2, \{\}, \ell),$$

In the latter case, it holds that $\sigma_1(x) = \text{val}^*(\sigma_2(x))$ for all $x \in \text{Var}$.

Finally, we restate the definition of $\text{decisions}_C: \mathcal{T} \rightarrow \{\perp\} \cup \mathcal{L}$ which returns the unique concretization decisions made by any trace,

$$\text{decisions}_C(\text{tr}) = \begin{cases} \ell & \text{if } \exists \sigma \in \mathcal{S}^*, \ell \in \mathcal{L}: \llbracket C \rrbracket^\dagger(\sigma_0, \text{tr}^\dagger, []) = (\sigma, \{\}, \ell), \\ \perp & \text{otherwise.} \end{cases}$$

B.3 Semantics of Program Transformation

Recall that the "replay" semantics of the program transformation, parametrised by a list of decisions $\ell \in \mathcal{L}$, are given by $\llbracket \cdot \rrbracket^\cup: \{\perp\} \uplus (\mathcal{S}^* \times \mathcal{T}^\dagger \times \mathcal{L}) \rightarrow \{\perp\} \uplus (\mathcal{S}^* \times \mathcal{T}^\dagger \times \mathcal{L})$. We have $\llbracket C \rrbracket^\cup = \llbracket C \rrbracket^\dagger$ except for the concretization command:

$$\begin{aligned} \llbracket x = \text{concretize}(E) \rrbracket^\cup(\sigma, \text{tr}, \ell) &= \text{if } (\text{typeerr}^\dagger(\sigma, E)) \text{ then } \perp \\ &\text{else if } (\exists G \in \mathbb{G}: \llbracket E \rrbracket^\dagger \sigma \in G \wedge \ell \neq []) \text{ then} \\ &\quad (\sigma[x \mapsto \ell(1), \mathbf{b} \mapsto \sigma(\mathbf{b}) \wedge (\ell(1) = \iota^\dagger(\llbracket E \rrbracket^\dagger \sigma))], \text{tr}, \ell.\text{removefirst}()) \\ &\text{else if } (\exists G \in \mathbb{G}: \llbracket E \rrbracket^\dagger \sigma \in G) \text{ then } (\sigma[x \mapsto \iota(\llbracket E \rrbracket^\dagger \sigma)], \text{tr}, \ell) \text{ else } \perp. \end{aligned}$$

Instead of using the concretized values of the input trace tr , $\llbracket \cdot \rrbracket^\cup$ consumes the pre-defined decisions ℓ and tracks mismatches at $\sigma(\mathbf{b}) \in \mathbb{B}$. By construction, following lemma holds.

LEMMA 7. For command C , trace $\text{tr} \in \mathcal{T}$, decision list $\ell \in \mathcal{L}$, and state $\sigma \in \mathcal{S}^*$, we have

$$\begin{aligned} \exists \sigma_1 \in \mathcal{S}^*: \llbracket C \rrbracket^\dagger(\sigma, \text{tr}^\dagger, []) = (\sigma_1, \{\}, \ell) &\iff \exists \sigma_2 \in \mathcal{S}^*: \llbracket C \rrbracket^\cup(\sigma[\mathbf{b} \mapsto 1], \text{tr}^\dagger, \ell) = (\sigma_2, \{\}, []) \\ &\quad \wedge \sigma_2(\mathbf{b}) = 1. \end{aligned}$$

In this case, it also holds that $\sigma_1(x) = \sigma_2(x)$ for all $x \in \text{Var} \setminus \{\mathbf{b}\}$.

This gave rise to two functions which formally define a sub-model. First, the membership test function $1_C^\ell: \mathcal{T} \rightarrow \mathbb{B}$ given by

$$1_C^\ell(\text{tr}) = \begin{cases} \sigma(\mathbf{b}) & \text{if } \exists \sigma \in \mathcal{S}^*: \llbracket C \rrbracket^\cup(\sigma_0[\mathbf{b} \mapsto 1], \text{tr}^\dagger, \ell) = (\sigma, \{\}, []), \\ 0 & \text{otherwise.} \end{cases}$$

Second, the density $p_C^\ell: \mathcal{T} \rightarrow [0, \infty)$ given by

$$p_C^\ell(\text{tr}) = \begin{cases} \sigma(\mathbf{p}) \cdot \sigma(\mathbf{w}) & \text{if } \exists \sigma \in \mathcal{S}^*: \llbracket C \rrbracket^\cup(\sigma_0[\mathbf{b} \mapsto 1], \text{tr}^\dagger, \ell) = (\sigma, \{\}, []), \\ 0 & \text{otherwise.} \end{cases}$$

B.4 Soundness proof

To construct the straight-line-program C^ℓ in Theorem 1, we give small-step operational semantics equivalent to $\llbracket \cdot \rrbracket^\cup$ with transitions $C \times \mathcal{A} \times C \rightarrow C \times \mathcal{A} \times C$ and final transitions $C \times \mathcal{A} \times C \rightarrow \mathcal{A} \times C$, where C is the set of commands and $\mathcal{A} = \mathcal{S}^* \times \mathcal{T}^\dagger \times \mathcal{L}$.

- Skip command

$$\overline{(\text{skip}, (\sigma, \text{tr}, \ell), SLP) \rightarrow ((\sigma, \text{tr}, \ell), SLP)}$$

- Assignment

$$\frac{\neg \text{typeerr}^\dagger(\sigma, E)}{(x = E, (\sigma, \text{tr}, \ell), SLP) \rightarrow ((\sigma[x \mapsto \llbracket E \rrbracket^\dagger \sigma, \text{tr}, \ell), (SLP; x = E))}$$

- Sequencing

$$\frac{(C_1, a, SLP) \rightarrow (C', a', SLP')}{(C_1; C_2, a, SLP) \rightarrow (C'; C_2, a', SLP')} \quad \frac{(C_1, a, SLP) \rightarrow (a', SLP')}{(C_1; C_2, a, SLP) \rightarrow (C_2, a', SLP')}$$

- If command

$$\frac{\neg \text{typeerr}^\dagger(\sigma, E) \wedge \llbracket E \rrbracket^\dagger \sigma \in \mathbb{B} \wedge \llbracket E \rrbracket^\dagger \sigma = \text{true}}{(\text{if } E \text{ then } C_1 \text{ else } C_2, (\sigma, \text{tr}, \ell), SLP) \rightarrow (C_1, (\sigma, \text{tr}, \ell), SLP)}$$

$$\frac{\neg \text{typeerr}^\dagger(\sigma, E) \wedge \llbracket E \rrbracket^\dagger \sigma \in \mathbb{B} \wedge \llbracket E \rrbracket^\dagger \sigma = \text{false}}{(\text{if } E \text{ then } C_1 \text{ else } C_2, (\sigma, \text{tr}, \ell), SLP) \rightarrow (C_2, (\sigma, \text{tr}, \ell), SLP)}$$

- While command

$$\overline{(\text{while } E \text{ do } C, (\sigma, \text{tr}, \ell), SLP) \rightarrow (\text{if } E \text{ then } (\text{while } E \text{ do } C) \text{ else skip}, (\sigma, \text{tr}, \ell), SLP)}$$

- Sample command

$$\frac{\llbracket S \rrbracket^\dagger \sigma \in \text{dom}(\text{tr}) \wedge \neg \text{typeerr}^\dagger(\sigma, S, d(E_1, \dots, E_n)) \wedge \text{tr}(\llbracket S \rrbracket^\dagger \sigma) \in \text{supp}^\dagger(d)}{(x = \text{sample}(S, d(E_1, \dots, E_n)), (\sigma, \text{tr}, \ell), SLP) \rightarrow ((\sigma[x \mapsto \text{tr}(\llbracket S \rrbracket^\dagger \sigma), \mathbf{p} \mapsto \sigma(\mathbf{p}) \cdot \text{pdf}_d^\dagger(\text{tr}(\llbracket S \rrbracket^\dagger \sigma), \llbracket E_1 \rrbracket^\dagger \sigma, \dots, \llbracket E_n \rrbracket^\dagger \sigma)], \text{tr} \setminus \llbracket S \rrbracket^\dagger \sigma, \ell), (SLP; x = \text{sample}(S, d(E_1, \dots, E_n))))}$$

- Score command

$$\frac{\llbracket S \rrbracket^\dagger \sigma \notin \text{dom}(\text{tr}) \wedge \neg \text{typeerr}^\dagger(\sigma, S, d(E_1, \dots, E_n), E_{n+1}) \wedge \llbracket E_{n+1} \rrbracket^\dagger \sigma \in \text{supp}^\dagger(d)}{(\text{score}(S, d(E_1, \dots, E_n), E_{n+1}), (\sigma, \text{tr}, \ell), SLP) \rightarrow ((\sigma[\mathbf{w} \mapsto \sigma(\mathbf{w}) \cdot \text{pdf}_d^\dagger(\llbracket E_{n+1} \rrbracket^\dagger \sigma, \llbracket E_1 \rrbracket^\dagger \sigma, \dots, \llbracket E_n \rrbracket^\dagger \sigma)], \text{tr}, \ell), (SLP; \text{score}(S, d(E_1, \dots, E_n), E_{n+1})))}$$

- Concretization

$$\frac{\neg \text{typeerr}^\dagger(\sigma, E) \wedge \exists G \in \mathbb{G}: \llbracket E \rrbracket^\dagger \sigma \in G[]^\dagger \wedge \ell \neq []}{(x = \text{concretize}(E), (\sigma, \text{tr}, \ell), SLP) \rightarrow ((\sigma[x \mapsto \ell(1), \mathbf{b} \mapsto \sigma(\mathbf{b}) \wedge (\ell(1) = \iota^\dagger(\llbracket E \rrbracket^\dagger \sigma)]), \text{tr}, \ell.\text{removefirst}()), (SLP; x = \ell(1)))}$$

$$\frac{\neg \text{typeerr}^\dagger(\sigma, E) \wedge \exists G \in \mathbb{G}: \llbracket E \rrbracket^\dagger \sigma \in G[]}{(x = \text{concretize}(E), (\sigma, \text{tr}, \ell), SLP) \rightarrow ((\sigma[x \mapsto \iota(\llbracket E \rrbracket^\dagger \sigma)], \text{tr}, \ell), (SLP; x = E))}$$

With standard induction arguments the equivalence of the small-step operational semantics and the denotational replace semantics $\llbracket \cdot \rrbracket^\cup$ can be shown:

LEMMA 8. For trace $\text{tr} \in \mathcal{T}$, decision list $\ell \in \mathcal{L}$, and state $\sigma \in \mathcal{S}^*$, it holds that

$$\begin{aligned} \llbracket C \rrbracket^\cup(\sigma_0[\mathbf{b} \mapsto 1], \text{tr}^\dagger, \ell) = (\sigma, \{\}, []) &\iff \exists n \geq 0, (C_i, a_i, SLP_i) \in C \times \mathcal{A} \times C, SLP_{n+1} \in C : \\ (C_0, a_0, SLP_0) &= (C, \sigma_0[\mathbf{b} \mapsto 1], \text{tr}^\dagger, \ell, \text{skip}) \\ \wedge \forall i = 0, \dots, n-1: &(C_i, a_i, SLP_i) \rightarrow (C_{i+1}, a_{i+1}, SLP_{i+1}) \\ \wedge (C_n, a_n, SLP_n) &\rightarrow ((\sigma, \{\}, []), SLP_{n+1}) \end{aligned}$$

In addition to this equivalence, the small-step semantics also construct a command which is a sequence of sample, score, and concretization commands (and a skip command at the beginning) as desired. Before continuing with the proof of Theorem 1, we establish one final necessary lemma, which characterises a type of determinism of the small-step semantics.

LEMMA 9. For commands $C, C_1, C_2, SLP, SLP_1, SLP_2 \in C$, traces $\text{tr}_1, \text{tr}_2, \text{tr}'_1, \text{tr}'_2 \in \mathcal{T}^\dagger$, decision lists $\ell, \ell_1, \ell_2 \in \mathcal{L}$ and states $\sigma_1, \sigma_2, \sigma'_1, \sigma'_2 \in \mathcal{S}^*$, assume $(C, (\sigma_i, \text{tr}_i, \ell), SLP) \rightarrow (C_i, (\sigma'_i, \text{tr}'_i, \ell_i), SLP_i)$ for $i = 1, 2$. If the program states σ_1 and σ_2 agree on untracked variables, i.e.,

$$\forall x \in \text{Var} \setminus \{\mathbf{p}, \mathbf{w}, \mathbf{b}\} : \sigma_1(x) \in \text{Val} \vee \sigma_2(x) \in \text{Val} \implies \sigma_1(x) = \sigma_2(x), \quad (5)$$

then the transitions coincide in the following way:

$$C_1 = C_2 \wedge \ell_1 = \ell_2 \wedge SLP_1 = SLP_2 \wedge \sigma'_1 \text{ agrees with } \sigma'_2 \text{ on untracked variables.}$$

The same holds for final transitions $(C, (\sigma_i, \text{tr}_i, \ell), SLP) \rightarrow ((\sigma'_i, \text{tr}'_i, \ell_i), SLP_i)$ for $i = 1, 2$.

PROOF. First, we note that for sequence commands and all final transitions except concretization, there is at most one applicable small-step rule. Thus, we have $C_1 = C_2$ and $SLP_1 = SLP_2$ for those commands, immediately. Further, the concretize command is the only one which modifies the decisions list ℓ and we have $\ell_1 = \ell_2$ for all other commands.

Next, we prove that $C_1 = C_2$ and $SLP_1 = SLP_2$ for the remaining commands and that σ'_1 agrees with σ'_2 on untracked variables via rule induction. We assume that (5) holds.

- *Skip command*: Follows immediately.
- *Assignment*: Only variable x is updated. If we assume that $\sigma'_1(x) = \llbracket E \rrbracket^\dagger \sigma_1 \in \text{Val}$, then by Lemma 5, for all variables $y \in \text{vars}(E)$ we have $\sigma_1(y) \in \text{Val}$. By (5), $\sigma_1(y) = \sigma_2(y)$ and thus, $\sigma'_1(x) = \llbracket E \rrbracket^\dagger \sigma_1 = \llbracket E \rrbracket^\dagger \sigma_2 = \sigma'_2(x)$, also. If we assume that $\sigma'_1(x) \notin \text{Val}$, then it follows similarly that we must have $\sigma'_2(x) \notin \text{Val}$.
- *Sequencing*: Both rules follow immediately from the induction assumption.
- *If command*: Since we have $\llbracket E \rrbracket^\dagger \sigma_i \in \mathbb{B} \subseteq \text{Val}$, as in the assignment case, it follows from Lemma 5 that $\llbracket E \rrbracket^\dagger \sigma_1 = \llbracket E \rrbracket^\dagger \sigma_2$. Therefore, $C_1 = C_2$ and $SLP_1 = SLP_2$ as required. The states do not change.
- *While command*: Follows from uniqueness of transition.
- *Sample command*: Only variable x is updated. Since the new values are *tracked* arrays the updated states σ'_1 and σ'_2 still agree on untracked values.
- *Score command*: Follows immediately as no variable updates are made.
- *Concretization*: If $\llbracket E \rrbracket^\dagger \sigma_1 \in G[] \subseteq \text{Val}$, then $\sigma'_2(x) = \llbracket E \rrbracket^\dagger \sigma_2 = \llbracket E \rrbracket^\dagger \sigma_1 = \sigma'_1(x) \in G[]$ by (5) and Lemma 5 as in the assignment case. Thus, the same transition is performed for $(\sigma_2, \text{tr}_2, \ell)$: $C_1 = C_2$, $SLP_1 = SLP_2$, and $\ell_1 = \ell = \ell_2$. If $\llbracket E \rrbracket^\dagger \sigma_1 \in G_1[]^\dagger \not\subseteq \text{Val}$, then $\llbracket E \rrbracket^\dagger \sigma_2 \notin \text{Val}$ (as otherwise implies that $\llbracket E \rrbracket^\dagger \sigma_1 = \llbracket E \rrbracket^\dagger \sigma_2 \in \text{Val}$ as before), such that $\llbracket E \rrbracket^\dagger \sigma_2 \in G_2[]^\dagger$. But then, $\sigma'_1(x) = \ell(1) = \sigma'_2(x) \in \text{Val}$, $\ell_1 = \ell.\text{removefirst}() = \ell_2$, $C_1 = C_2$ and $SLP_1 = SLP_2$, also.

□

THEOREM 1. Let C be a command and let $\text{dr} \in \mathcal{T}$ with shape $s \in \mathbb{S}$ be the decision representative of the sub-model with $\ell = \text{decisions}_C(\text{dr}) \neq \perp$. Then, p_C^ℓ and 1_C^ℓ are measurable functions. Further, there exists a command C^ℓ which is a sequence of assignment, sample and score commands, such that

$$\forall \text{tr} \in \mathcal{T}: 1_C^\ell(\text{tr}) = 1 \implies p_C^\ell(\text{tr}) = p_C(\text{tr}) = p_{C^\ell}(\text{tr}) \wedge \text{tr} \in \prod_{(\alpha, A) \in s} A.$$

In words, if tr belongs to the same sub-model as dr , then p_C^ℓ computes the correct density, tr has the same shape as dr , and the sub-model can indeed be represented by a straight-line-program C^ℓ .

PROOF. First, since $\mathcal{D}^\dagger$ is a ω -cpo and because append, removefirst, list access, the isomorphism $\iota^\dagger = \iota \circ \text{val}$, equality test, and logical conjunction are all measurable functions, it follows that the semantic functions $\llbracket \cdot \rrbracket^\dagger \in \mathcal{D}^\dagger$ and $\llbracket \cdot \rrbracket^\cup \in \mathcal{D}^\dagger$ are well-defined for the same arguments as in the proof of Lemma 4. Further, as an immediate consequence, we have that p_C^ℓ and 1_C^ℓ are measurable functions.

Let $\text{tr} \in \mathcal{T}$ be a trace with $1_C^\ell(\text{tr}) = 1$, thus

$$\exists \sigma \in \mathcal{S}^*: \llbracket C \rrbracket^\cup(\sigma_0[\mathbf{b} \mapsto 1], \text{tr}^\dagger, \ell) = (\sigma, \{\}, []) \text{ and } \sigma(\mathbf{b}) = 1.$$

By Lemma 7, we have $\llbracket C \rrbracket^\dagger(\sigma_0, \text{tr}^\dagger, []) = (\sigma, \{\}, \ell)$ and by Lemma 6 also $\llbracket C \rrbracket(\sigma_0, \text{tr}) = (\sigma', \{\})$ for some state σ' such that $\sigma'(x) = \text{val}^*(\sigma(x))$ for all $x \in \text{Var} \setminus \{\mathbf{b}\}$. By definition of the density functions, we have $p_C^\ell(\text{tr}) = \sigma(\mathbf{p}) \cdot \sigma(\mathbf{w}) = \sigma'(\mathbf{p}) \cdot \sigma'(\mathbf{w}) = p_C(\text{tr})$.

We conclude the proof with following coupling argument. Let $(C_i^1, (\sigma_i^1, \text{tr}_i^1, \ell_i^1), \text{SLP}_i^1), i = 1, \dots, n_1$, and $(C_i^2, (\sigma_i^2, \text{tr}_i^2, \ell_i^2), \text{SLP}_i^2), i = 1, \dots, n_2$, be the sequences of Lemma 8 produced by dr and tr , respectively, with end states $((\sigma^{\text{dr}}, \{\}, []), \text{SLP}_{n_1+1}^1)$ and $((\sigma, \{\}, []), \text{SLP}_{n_2+1}^2)$.

Applying Lemma 9 inductively yields $n_1 = n_2$, $C_i^1 = C_i^2$, $\ell_i^1 = \ell_i^2$, $\text{SLP}_i^1 = \text{SLP}_i^2$, and σ_i^1 agrees with σ_i^2 on untracked variables, for all $i = 1, \dots, n$. Thus, also the last produced command are equal $\text{SLP}_{n+1}^1 = \text{SLP}_{n+1}^2$ and σ agrees with σ^{dr} on untracked variables. We define $C^\ell = \text{SLP}_{n+1}^1$.

Furthermore, let $\llbracket \text{SLP}_i^2 \rrbracket(\sigma_0, \text{tr}) = (\sigma'_i, \text{tr}'_i)$. By rule induction it is straightforward to show that $\forall x \in \text{Var} \setminus \{\mathbf{b}\}: \sigma'_i(x) = \text{val}^*(\sigma_i^2(x))$. This, also holds for $C^\ell = \text{SLP}_{n+1}^2$ and we conclude that $p_{C^\ell}(\text{tr}) = \sigma'_{n+1}(\mathbf{p}) \cdot \sigma'_{n+1}(\mathbf{w}) = \sigma(\mathbf{p}) \cdot \sigma(\mathbf{w}) = p_C^\ell(\text{tr})$.

Lastly, assume that $C_i = C_i^1 = C_i^2 = (x = \text{sample}(S, d(E_1, \dots, E_n)))$. Since $\neg \text{typeerr}(\sigma_i^k, S)$ holds, by Lemma 5, we have $\forall x \in \text{vars}(S): \sigma_i^k(x) \in \text{Val}$ for both $k = 1, 2$. As σ_i^1 and σ_i^2 agree on untracked variables, $\sigma_i^1(x) = \sigma_i^2(x)$ for all $x \in \text{vars}(S)$, and thus $\llbracket S \rrbracket^\dagger \sigma_i^1 = \llbracket S \rrbracket^\dagger \sigma_i^2$. Therefore, the trace updates $\text{tr}_i^k \setminus \llbracket S \rrbracket^\dagger \sigma_i^k$ remove the same entry $(\llbracket S \rrbracket^\dagger \sigma_i^k, \text{supp}^\dagger(d))$ from the trace shapes. Both end states contain empty traces $\{\}$ having the same empty shape. As the trace shapes change equally at sample commands, we can reason backwards in the sequence and conclude that also the initial traces tr and dr must have had the same shape, i.e., $\text{tr} \in \prod_{(\alpha, A) \in s} A$.

□

C Remark on Connection to Measure Semantics

In this section, we briefly describe a connection of the density semantics to denotational semantics defined by *sub-probability kernels* following Lee et al. [28, Section 4.2 and 4.3].

Let $\mathcal{P}_{\leq 1}(\mathcal{S} \times \mathcal{T}, \mathcal{S} \times \mathcal{T})$ be the set of all probability kernels from $\mathcal{S} \times \mathcal{T}$ to $\mathcal{S} \times \mathcal{T}$. These are functions $k: (\mathcal{S} \times \mathcal{T}) \times \Sigma_{(\mathcal{S} \times \mathcal{T})} \rightarrow [0, 1]$, such that $(\sigma, \text{tr}) \mapsto k((\sigma, \text{tr}), A)$ is a measurable function for all $A \in \Sigma_{\mathcal{S} \times \mathcal{T}}$ and $A \mapsto k((\sigma, \text{tr}), A)$ is a sub-probability measure on $\mathcal{S} \times \mathcal{T}$ for all $(\sigma, \text{tr}) \in \mathcal{S} \times \mathcal{T}$.

It is possible to adapt the denotational measure semantics of Lee et al. [28, Fig. 5.] in the same way as we have adapted the denotational density semantics of Lee et al. [28, Fig. 6.] in Appendix A. Let $\llbracket C \rrbracket_{\mathcal{K}} \in \mathcal{P}_{\leq 1}(\mathcal{S} \times \mathcal{T}, \mathcal{S} \times \mathcal{T})$ denote this semantic function. This allows us to define the unnormalised posterior distribution of command C directly:

$$\mu_C(A) = \int_{\mathcal{S} \times A} \sigma(\mathbf{w}) \llbracket C \rrbracket_{\mathcal{K}}((\sigma_0, \{\}), d(\sigma, \text{tr})).$$

The proofs of [28, Theorem 4.9, Corollary 4.10] may be extended to our setting to show that

$$\mu_C(A) = \int_A p_C(\text{tr}) \mu_{\mathcal{T}}(d\text{tr}).$$

In these proofs the locality property of set $\mathcal{D}$ is necessary, see Appendix A.4.